Identity as Accountability Chain

A Post-Quantum Architecture for Persistent Identity
Across Human, Machine, and AI Agent Substrates

Sylvain Cormier

Paraxiom Technologies Inc.

sylvain@paraxiom.org

May 22, 2026

We propose a formal model in which *identity* is not a primitive but a layered accountability chain that accretes from a founding individuation event and persists as long as cryptographic, relational, and civic verification can be reconstructed. The model unifies biological, machine, and artificial-intelligence cases under a single schema with six accretion layers and a four-tier permanence hierarchy for substrate-level identifiers.

We state and prove seven results that bear directly on engineering: (i) Founding uniqueness — every identity locus has exactly one origin event; (ii) Tier monotonicity — the permanence of an identity claim is non-increasing in the tier of its substrate identifier; (iii) Stratum well-foundedness — every scoped or agent identity traces in finite steps to a person- or organization-rooted identity; (iv) LLM instance non-identity — two LLM instances with identical weights but disjoint deployment IDs are distinct identity loci; (v) Cryptographic survival — only post-quantum-signed identities remain verifiable past a cryptographically-relevant quantum computer (CRQC) event; (vi) Crypto-shredding completeness — master-key deletion renders all envelope-encrypted credentials computationally inaccessible without recourse to unpinning; and (vii) Verification-time accountability — the reference `verify_chain` procedure terminates correctly in a finite number of steps. The seven theorems are accompanied by a Lean 4 formalization [24] (Mathlib v4.27.0) that builds cleanly with zero sorries: four of the seven (founding uniqueness, stratum well-foundedness, LLM instance non-identity, verification-time accountability) are kernel-checked deductively from the model primitives; the remaining three (tier monotonicity, cryptographic survival, crypto-shredding completeness) are kernel-checked at the chain-reduction layer relative to named cryptographic and probabilistic axioms (FIPS 203/204/205 EUF-CMA, Shor against classical discrete log, AES-256-GCM IND-CPA, and an empirical permanence-tier rate ordering) — the same convention as EasyCrypt and CryptoVerif. The repository pins the same Lean/Mathlib versions as Paraxiom's other formal-verification projects.

The treatise also names four distinct sources from which identity content arises — externally witnessed, self-asserted, cumulatively earned, and algorithmically derived — and formalizes self-assertion as a first-class entry in the accountability chain, signed by the locus's own key. A companion subsection scopes non-consensual third-party identity (data-broker dossiers, surveillance records, algorithmic scoring labels) explicitly out of the accountability-chain primitive, names the asymmetry against user-controlled disclosure, and points to the regulatory layer (GDPR Art. 17, CCPA §1798.105) that addresses it. A hypergraph generalization of the binary relational graph promotes higher-arity identity facts (joint authorship, committee-ratifies-charter, multi-party agreements) to first-class status, with the *identity-state hash* $h_{\mathcal{I}}(t) = H(\mathcal{F} \,\|\, \mathcal{A}_{\leq t} \,\|\, \mathcal{H}_{\leq t})$ addressing identity by current state rather than by a fixed handle. The treatise covers six founding-event domains: human, machine, container, process, LLM substrate, LLM instance, and physical good (NFC/RFID-anchored, NFT-bound) — the last is integrated with the production NFT-identity lane (Aura, Arianee, VeChain, ERC-6551).

We survey the state of digital identity as of May 2026, noting that no major civic identity system runs post-quantum signatures in production despite regulatory deadlines of 2030–2035, and that AI agent identity is a standards vacuum. A dedicated positioning subsection tabulates the model-layer primitives this treatise contributes over and above the W3C VC 2.0 (Recommendation, 15 May 2025) and DIDs v1.1 (Candidate Recommendation Snapshot, 5 March 2026) stack: post-quantum signatures, strata, hypergraph state, physical-good integration, accountability tracing, and crypto-shredding are not in the published W3C corpus.

We then present the Paraxiom architecture: a three-tier system combining on-chain post-quantum primitives (QuantumHarmony, with SPHINCS+ and Falcon-512), IPFS for bulk encrypted payloads, and Postgres+Redis mirrors for query-layer access. The architecture is grounded in five identity strata (S1–S5) covering humans, scoped delegations, vault credentials, agent instances, and cross-tenant aggregates, with provision for future extensions to swarm and protocol identities.

The treatise concludes that identity is engineering, not philosophy: the accountability chain does not extend itself; the post-quantum signatures do not sign themselves. What we build now determines whether identity remains a coherent concept past 2040.

Keywords: digital identity, post-quantum cryptography, accountability, agent identity, substrate attestation, decentralized identity, SPHINCS+, Falcon-512, Substrate, IPFS, eIDAS, AAMVA, MCP, verifiable credentials.

Contents

1. Introduction

1.1. Premise

A governance application requires an "identity primitive." A blockchain pallet needs a stratum field. A credential vault needs a user. Before we choose data structures, we must answer a prior question: what *is* identity, and when does it begin?

This treatise advances a single claim:

> *Identity is not a primitive. It is a layered accountability chain that accretes from the moment a locus of action becomes individuated, and persists exactly as long as the chain*

can verify continuity. When the chain breaks, identity dissolves — regardless of whether the substrate (body, hardware, weights file) is still running.

This claim holds for humans, for processes, for AI agents, for swarms, for organizations. The substrate differs; the accretion logic is identical. We will state this formally (Theorems 2.2 and 2.3), prove that it implies seven engineering invariants (Theorems 2.8, 5.2, 6.2, 7.3, 9.1, 11.1 and 11.3), and demonstrate that the Paraxiom architecture realizes them.

1.2. Why now

Three forces drive this work to operational urgency in 2026.

Post-quantum cryptography becomes load-bearing. NIST published ML-KEM, ML-DSA, and SLH-DSA as FIPS standards on 13 August 2024 [1, 2, 3]. The U.S. NSA's CNSA 2.0 prohibits classical algorithms in National Security Systems by 2035. The EU joint PQC roadmap (June 2025) targets identity-wallet PKI migration by the same horizon. Yet, as of May 2026, no major civic identity system operationally runs post-quantum signatures in production. The window in which a classical signature today can be forged retroactively by a sufficient quantum computer is the window in which the entire pre-CRQC accountability chain collapses.

AI agent identity is an open standards vacuum. The Linux Foundation's Agentic AI Foundation launched in December 2025 with contributions from Anthropic (MCP), OpenAI (AGENTS.md), and Block (Goose). MCP added a server-identity primitive in November 2025 [6]. Google's A2A protocol uses workload-identity federation [7]. Cloudflare-backed IETF Web Bot Auth proposes signed HTTP requests. No standard exists for model substrate attestation: cryptographically proving that a given output originated from a specific weights binary on a specific hardware root. The field is where SSI was in 2018.

Civic identity is fragmenting. EU eIDAS 2.0 mandates wallet deployment by 24 December 2026 [5]. US mDL proliferates state by state with no federal coordination. The UK GOV.UK One Login operates uncertified against its own trust framework. Canada lacks a federal equivalent to EUDI. The conditions for an architecture-of-record are present.

1.3. A note on the ontology

This architecture treats identity as a *conditioned phenomenon*, not an essence. There is no soul-primitive in the formal model; the layers of Section 3 are accretions on a founding event, and the chain merely witnesses what arises and passes. The crypto-shredding theorem (Theorem 11.1) is the formal counterpart of release: a master-key deletion renders the ciphertexts computationally inaccessible while the chain still holds the hashes — the meaning is gone but the trace persists as record. The dissolution semantics of Section 11 names the inevitability of ending.

We make no claim that identity is real in any deep sense. We claim only that, while identity manifests in the world, it should be possible to verify what is being claimed and by whom. The infrastructure is a humble one: it does not capture an essence because there is no essence to

capture; it makes verifiable the conditioned arising and passing of accountable loci. Karma operates whether or not there is a self; the accountability chain is the cryptographic witness to dependent origination, not a metaphysical foundation.

1.4. Method

Sections 2 to 7 and 9 develop the formal model. Section 10 surveys the deployed state of digital identity in May 2026. Section 11 presents the engineering realization, with proofs of crypto-shredding completeness and verification-time accountability enforcement. Sections 13 and 14 discuss scenario projections and concluding remarks. Appendices give detailed proofs, pallet API specifications, and sources.

2. Formal model of identity

We begin by stating the model abstractly. Subsequent sections specialize to biological, machine, agent, and LLM cases.

2.1. Primitive sets

Fix the following primitive sets:

- $\mathbb{T}$ — time, totally ordered.
- $\mathbb{K}$ — cryptographic keypairs over some signature scheme Σ, with operations $\mathsf{Sign}_{sk}, \mathsf{Verify}_{pk}$ satisfying EUF-CMA security.
- $\mathbb{C}$ — claim content (arbitrary byte-strings).
- $\mathbb{S}$ — substrates: a typed union of biological organisms, physical hosts, container images, weights blobs, and similar individuated material configurations.
- $\mathbb{L} = \{L_0, L_1, L_2, L_3, L_4, L_5\}$ — the six accretion layers defined in Section 3.

2.2. Definitions

Definition 2.1 (Founding event)**.** A *founding event* is a triple $\mathcal{F} = (s, t_0, \pi)$ where $s \in \mathbb{S}$ is a substrate, $t_0 \in \mathbb{T}$ is the event timestamp, and π is a finite set of witnesses (the empty set is permitted but reduces verifiability). The founding event marks the transition of s from a non-individuated state to an individuated locus capable of accreting claims.

Definition 2.2 (Accretion chain)**.** An *accretion chain* relative to founding event $\mathcal{F} = (s, t_0, \pi)$ is a finite sequence

$$\mathcal{A} = (a_1, a_2, \ldots, a_n), \quad a_i = (L_i, t_i, c_i, \sigma_i, \rho_i)$$

where $L_i \in \mathbb{L}$ is the layer tag, $t_i \geq t_0$ is the claim timestamp, $c_i \in \mathbb{C}$ is the claim content, σ_i is either a signature in $\mathbb{K}$ or a relational witness, and $\rho_i \in \{1, \ldots, i-1\} \cup \{\perp\}$ is an optional reference to a prior claim providing context. The sequence is monotone in time: $i < j \Rightarrow t_i \leq t_j$.

Definition 2.3 (Identity locus)**.** An *identity locus* is a quadruple

$$\mathcal{I} = (\mathcal{S}, \mathcal{F}, \mathcal{A}, \mathcal{R})$$

where $\mathcal{S} \in \mathbb{S}$ is its substrate, $\mathcal{F}$ is its founding event, $\mathcal{A}$ is its accretion chain, and $\mathcal{R} \subseteq \mathbb{I} \times \mathbb{I}$ is its relational graph with other identity loci (where $\mathbb{I}$ is the universe of all identity loci).

Definition 2.4 (Verifiable identity)**.** An identity locus $\mathcal{I}$ is *verifiable at time* t iff, for every layer-3 (cryptographic) claim $a_i \in \mathcal{A}$ with $t_i \leq t$, there exists a key $\mathsf{KDF}(\mathcal{I}, t_i) \in \mathbb{K}$ such that $\mathsf{Verify}(\mathsf{KDF}(\mathcal{I}, t_i), c_i, \sigma_i) = 1$ and the substrate $\mathcal{S}$ retains continuity through the interval $[t_0, t]$.

2.3. Axioms

We posit three axioms that scope the model.

Axiom 2.5 (Founding precedence)**.** For every identity locus $\mathcal{I}$, every claim $a_i \in \mathcal{A}$ satisfies $t_i \geq t_0$, where t_0 is the founding timestamp.

Axiom 2.6 (Substrate continuity or explicit succession)**.** An identity locus persists from time t_a to $t_b > t_a$ iff either (i) its substrate $\mathcal{S}$ remains individuated and physically continuous over $[t_a, t_b]$, or (ii) there exists a successor mapping $\Phi : \mathcal{I} \to \mathcal{I}'$ signed by an authority A appearing in $\mathcal{R}$, recorded at some $t \in [t_a, t_b]$.

Axiom 2.7 (Layer independence)**.** The six layers in $\mathbb{L}$ are pairwise distinguishable: no claim a_i in an accretion chain may simultaneously have two layer tags. The failure modes of different layers are statistically independent absent explicit coupling (e.g., a single physical device holding both Layer 3 and Layer 4 artifacts).

Theorems 2.5 to 2.7 imply that identity is a well-defined object: it has a beginning, a domain of persistence, and a coordinate system in $\mathbb{L}$.

2.4. First theorem: founding uniqueness

Theorem 2.8 (Founding uniqueness)**.** *For every identity locus $\mathcal{I}$, exactly one founding event $\mathcal{F}$ exists.*

Proof. By construction (Theorem 2.3), $\mathcal{I}$ specifies a single $\mathcal{F}$. Suppose for contradiction that two distinct events $\mathcal{F}_1, \mathcal{F}_2$ both qualify. By Theorem 2.5 all claims in $\mathcal{A}$ must precede neither. If $t_1 < t_2$, then $\mathcal{F}_2$ would violate the founding precedence axiom with respect to claims indexed near t_1; if $t_1 = t_2$, both events name the same individuation, hence $\mathcal{F}_1 = \mathcal{F}_2$. Contradiction. ■

Remark 2.9. Theorem 2.8 is the formal restatement of the engineering principle that an identity in `identity-registry` [9] must have a uniquely-recorded `created_at_block` and `created_by_tx_hash`. An identity record without a verifiable origin transaction is malformed.

2.5. Sources of identity content

Theorem 2.8 establishes *when* identity begins; it does not establish *who decides what an identity is*. The model above could be read as treating identity as a thing that arrives at a substrate from outside — a witness signs, a manufacturer fuses, a deployer assigns. This is not the complete picture. We name four distinct sources from which identity content arises, and posit that a complete identity is a composition of all four; no source has presumptive priority over the others, and the chain records which source contributed which claim.

Externally witnessed. A claim attested by a party other than the locus itself: the birthing parent and attending clinicians at the founding event, a KYC issuer's bundle, a hardware manufacturer's endorsement key. These claims populate L_2 (relational) and L_4 (civic).

Self-asserted. A claim signed by the locus itself with its own L_3 key, asserting content about itself: a chosen name, a pronoun set, a role title, a scope statement, a pseudonymous handle. We distinguish self-assertion from L_1 narrative: L_1 asserts *continuity* ("I am the same entity that did X"), while self-assertion asserts *content* ("I am Alex"; "my pronouns are …"; "I act as a code reviewer for repository R").

Cumulatively earned. A claim that accrues over time through behavior: reputation tokens in a body, signed attestations of completed work, settled-and-not-disputed actions. Populates L_5.

Algorithmically derived. A claim computed from substrate state without intentional choice: a weights hash, a deployment identifier, a TPM-attested measurement. Populates L_0.

The architecture records all four kinds without imposing a global precedence. Different verifiers attach different evidentiary weight to each source for different actions: a civic eligibility check requires L_4; a credential-vault interaction requires the locus's own L_3 signature; an arbitrary self-asserted name is sufficient for routine social interaction but does not unlock jurisdictional acts. The user-controlled disclosure principle (Theorem 12.1) is the formal counterpart: the holder chooses which sources to reveal per action; relying parties decide what weight to assign.

Definition 2.10 (Self-assertion)**.** A *self-assertion* is a claim $a_i = (L_i, t_i, c_i, \sigma_i, \rho_i) \in \mathcal{A}$ in which the signature is produced by the locus's own L_3 key, $\sigma_i = \text{Sign}_{sk(\mathcal{I})}(c_i)$, and the claim content c_i asserts a property of $\mathcal{I}$ itself. Self-assertions are recorded with the same primitives as any other claim; the chain adjudicates only their cryptographic well-formedness, not their external validity.

Remark 2.11. Theorem 2.10 formalizes that a locus has its own voice in its accountability chain. The chain is not a record kept *about* the locus by others; it is a record kept *with* the locus, in which the locus's signed claims about itself are first-class entries. The architecture's neutrality on what may be asserted is the formal counterpart of user-controlled disclosure: the locus chooses what to claim and what to reveal; the chain witnesses the choice; relying parties decide what weight to assign.

The hypergraph generalization of the relational structure and the *identity-state hash* primitive are developed formally in Section 2.7.

2.6. Non-consensual third-party identity (out of scope)

The four sources of Section 2.5 share a precondition: the locus is a co-author of its own chain — either by signing claims itself, by being knowably attested by named witnesses, by accumulating behavior on a chain it can read, or by being the substrate to which an algorithmic identifier is bound. A distinct and consequential class of identity-shaped records does not satisfy this precondition. We name the class explicitly, scope the model against it, and identify the regulatory layer that addresses it.

The class. *Non-consensual third-party identity* denotes records about a locus, maintained by parties other than the locus, without the locus's participation, signature, or read access. Canonical instances include:

- **Commercial dossiers**: data-broker files (Acxiom, Experian, LexisNexis), advertising identifiers (IDFA, AdID, browser fingerprints), customer-lifetime-value scores, shadow profiles maintained on non-users.
- **Surveillance records**: targeted-individual profiles (e.g., Pegasus), persona-tracking and selector databases, watchlist entries, biometric enrollment without informed consent (e.g., Worldcoin's iris incentive scheme, certain border-control programs).
- **Algorithmic labels**: credit scores, insurance-risk scores, social-credit-style state scoring, ML-classifier verdicts ("bot," "high-risk traveler," "inauthentic content") that become operationally identity-bearing.
- **OSINT / doxing compilations**: aggregated open-source intelligence dossiers, leak-database entries, hostile correlations across breached identifiers.
- **Coerced enrollment**: identity records produced under conditions that vitiate consent (refugee-status biometrics, conscript records, mandatory enrollment under sanction).

Scope decision. A record of this class is *not* an accountability chain in the sense of Theorem 2.2. The chain is co-authored by definition: claims are added either by the locus itself (Theorem 2.10) or by named witnesses whose attestations the locus can read, verify, and contest. A record kept about a locus that the locus cannot access satisfies none of these conditions. Treating such records as accountability chains would collapse the analytical distinction between the locus's own chain and records held against it by adversaries; the seven results of this treatise (and in particular stratum well-foundedness, Theorem 6.2) depend on co-authorship at S1.

The user-controlled disclosure asymmetry. The user-controlled disclosure principle (Theorem 12.1) governs disclosure *from* chains the locus co-authors. It has no operative counterpart over non-consensual third-party records: the locus cannot choose what is revealed from a record it does not hold, cannot read, and may not know exists. This asymmetry is a structural property of the present architecture; closing it is not an engineering problem internal to the chain but a regulatory and political problem external to it.

Regulatory layer. Remedies for non-consensual third-party identity exist outside the chain. GDPR Article 17 (right to erasure) and CCPA §1798.105 (right to deletion) compel deletion of personal data on request; fair-credit-reporting acts in multiple jurisdictions compel disclosure and dispute of scoring records; sectoral regimes (e.g., the EU AI Act for high-risk classifiers, ePrivacy for tracking identifiers) impose transparency and consent obligations. These regimes restrict third-party records; they do not produce accountability chains, and the architecture neither implements nor displaces them. A locus engaged in a dispute with a data broker, an employer, an intelligence agency, or a scoring authority operates against that party under the applicable regulatory regime, not through the chain.

Research directions. Closing the asymmetry is an open problem. Active directions include differential-privacy-preserving public registries (in which the existence and aggregate properties of adversarial records can be audited without revealing individual content), regulatory transparency-by-default for data brokers and high-impact classifiers, and adversarial-disclosure auditing that produces verifiable evidence of unauthorized profiling. We commit to naming the class so that relying parties, regulators, and loci themselves treat it correctly; we do not commit the architecture to a solution.

2.7. Hypergraph identity state

The relational graph $\mathcal{R}$ in Theorem 2.3 is *binary*: a relation $\mathcal{R} \subseteq \mathbb{I} \times \mathbb{I}$ between pairs of identity loci. Binary relations adequately model parent-child stratum links, peer attestations, and bilateral agreements — and these account for most chain primitives in Section 11. They do not adequately model the intrinsically higher-arity relationships in which identity is also constituted: a paper by n co-authors is a single 4-ary or 7-ary fact, not a quadratic in $\binom{n}{2}$ pairs; a committee that ratifies a charter binding an organization is a 3-ary fact (committee, charter, organization); a treaty among

five sovereigns is a 5-ary fact. The *hypergraph generalization* promotes the relational structure to the natural domain for these facts.

Definition 2.12 (Identity hypergraph)**.** An identity hypergraph over an identity universe $\mathbb{I}$ is a pair $\mathcal{H} = (V, E)$ where $V \subseteq \mathbb{I}$ is the vertex set and $E \subseteq 2^V \setminus \{\emptyset\}$ is the hyperedge set. Each hyperedge $e \in E$ is annotated with a tuple $(t_e, \tau_e, \mu_e, \Sigma_e)$ in which t_e is the time of formation, τ_e is the hyperedge kind (e.g. joint-authorship, ratifies-charter, multi-party-signature, cross-body-endorsement), μ_e is the optional payload (e.g. the charter hash, the document content), and Σ_e is the set of cryptographic signatures attesting the hyperedge (one per participating vertex when the kind requires unanimity).

Remark 2.13. The binary relation $\mathcal{R}$ is the restriction of $\mathcal{H}$ to hyperedges of cardinality two. Every result in this treatise that refers to $\mathcal{R}$ remains valid under $\mathcal{H}$ by considering only its $\binom{V}{2}$ projection; the hypergraph extension is strictly generative.

We give three concrete examples.

Joint authorship. An academic paper with four co-authors $\{\mathcal{I}_1, \ldots, \mathcal{I}_4\}$ is a single hyperedge $e_{\text{auth}} = \{\mathcal{I}_1, \mathcal{I}_2, \mathcal{I}_3, \mathcal{I}_4\}$ of kind joint-authorship, payload the paper's hash, signatures the four co-authors' Falcon-512 signatures over the hash. This is one fact, not six pairwise facts; the binary projection loses the joint character.

Committee ratifies charter. A G_2 committee body B_c ratifies a charter χ that binds a G_3 organization B_o. The fact is a 3-uniform hyperedge $\{B_c, \chi, B_o\}$ of kind ratifies-charter; payload the on-chain charter CID; signatures from the committee's authorizing members.

Multi-party signed agreement. A five-party agreement on a shared protocol — e.g., five sovereign G_5 bodies acknowledging the same cryptographic standard — is a 5-ary hyperedge with the agreement's hash as payload and one signature per party in Σ_e. A binary representation would require $\binom{5}{2} = 10$ pairwise treaties and could not distinguish a bilateral agreement from a multilateral one.

Definition 2.14 (Identity-state hash)**.** Let $\mathcal{I}$ be an identity locus with founding event $\mathcal{F}$, accretion chain prefix $\mathcal{A}_{\leq t}$, and hypergraph restriction $\mathcal{H}_{\leq t}(\mathcal{I}) \subseteq \mathcal{H}$ consisting of the hyperedges incident on $\mathcal{I}$ at time t. The *identity-state hash* of $\mathcal{I}$ at t is

$$h_{\mathcal{I}}(t) \;=\; H\big(\, \mathcal{F} \parallel \mathrm{Ser}(\mathcal{A}_{\leq t}) \parallel \mathrm{Ser}(\mathcal{H}_{\leq t}(\mathcal{I}))\,\big),$$

where H is a collision-resistant hash function (Blake2-256 in the chain implementation) and $\mathrm{Ser}(\cdot)$ is a canonical serialization that fixes the order of accretion entries by timestamp and the order of hyperedges by a lexicographic key over their participants.

Remark 2.15 (Properties)**.** Three immediate properties of Theorem 2.14: (i) *Binding*: by collision resistance of H, no computationally-bounded adversary can produce a distinct state $(\mathcal{F}', \mathcal{A}', \mathcal{H}')$ that hashes to the same value. (ii) *Evolution monotonicity*: $h_{\mathcal{I}}(t)$ changes exactly when the chain accretes a new claim or when a hyperedge incident on $\mathcal{I}$ is added or revoked; key rotation alone does not change it (since $sk(\mathcal{I})$ is not in the preimage), and substrate change alone does

not change it (since $\mathcal{S}$ is not in the preimage either). (iii) *Selective-disclosure compatibility*: any subset $S \subseteq \mathcal{H}_{\leq t}(\mathcal{I})$ of incident hyperedges can be presented to a verifier along with a merkle path against $h_{\mathcal{I}}(t)$; the verifier confirms membership of S in $\mathcal{I}$'s state at t without learning the remaining hyperedges. The merkle construction over $\mathcal{H}_{\leq t}(\mathcal{I})$ is implementation-defined; the identity-state hash binds it.

Positioning against W3C RDF graphs. The W3C Verifiable Credentials Data Model 2.0 [11] represents claims as RDF triples (subject, predicate, object), which form a binary edge-labelled graph. RDF can encode higher-arity facts via *reification* or *n-ary relation patterns*, but these are not native primitives: a 4-author paper expressed as RDF becomes a blank-node intermediary connecting four authorship-statement triples, losing the single-fact character. The hypergraph primitive of Theorem 2.12 is the direct representation; the W3C primitive is its emulation.

Implementation status. The v0.1 and v0.2 chain operate on the binary $\mathcal{R}$ and do not realize the hypergraph extension; the identity-state hash is defined model-side but not exposed via an extrinsic. The v0.3 chain ships a `pallet-hypergraph-state` implementing $\mathcal{H}$ as a content-addressed hyperedge store with $h_{\mathcal{I}}$ derived from on-chain state. Forward compatibility with v0.2 is preserved: every v0.2 relational fact appears as a 2-ary hyperedge in $\mathcal{H}$ without translation.

2.8. Formalization status

The seven principal theorems of this treatise are accompanied by a companion Lean 4 formalization [24] that builds against Mathlib v4.27.0 (the same pin as Paraxiom's other formal-verification projects). The current build produces zero sorries, zero kernel errors, and the kernel-checked statements correspond directly to the theorem identifiers cited inline below. Table 1 maps the theorems to their Lean identifiers and notes whether each is fully deductive from the model primitives or chain-reduction-checked relative to a named external axiom.

Explicit axiom statements. The four axioms named in Table 1 stand for the following claims:

tier_monotonicity_axiom For any survival-probability function $P_{\text{survive}} : \mathbb{T}_{\text{tier}} \to \mathbb{R}$ over the four-tier hierarchy of Section 5, the function is non-increasing in the tier index. The axiom encodes an empirical claim about hardware-replacement rate, installation-reset rate, configuration-rotation rate, and process-restart rate; the underlying probability model is not formalized here.

euf_cma_pq_secure_under_crqc For any post-quantum signature scheme Σ standardized under FIPS 203, 204, or 205 [1, 2, 3], and any polynomial-time quantum adversary $\mathcal{A}$ with access to a chosen-message oracle, the EUF-CMA forgery advantage of $\mathcal{A}$ against Σ is negligible in the post-quantum security parameter $\lambda \geq 128$.

Treatise theorem	Lean identifier	Verification mode
2.8 Founding uniqueness	`founding_uniqueness`	Deductive from `IdentityLocus` construction
5.2 Tier monotonicity	`tier_monotonicity`	Chain-level reduction; relies on tier_monotonicity_axiom (empirical permanence-rate ordering)
6.2 Stratum well-foundedness	`stratum_wellfoundedness`	Deductive (Nat strong induction over chain block height); see Appendix D
7.3 LLM instance non-identity	`llm_instance_non_identity`	Deductive from record-equality on `LLMInstance`
9.1 Cryptographic survival	`cryptographic_survival`	Chain-level reduction; relies on euf_cma_pq_secure_under_crqc and shor_breaks_classical_under_crqc
11.1 Crypto-shredding completeness	`crypto_shredding_completeness`	Chain-level reduction; relies on aes_256_gcm_ind_cpa
11.3 Verification-time accountability	`verification_time_accountability`	Deductive (via `stratum_wellfoundedness`)

Table 1: Mapping of treatise theorems to Lean identifiers and verification modes. Four of the seven theorems are kernel-checked deductively against the model primitives; the remaining three are kernel-checked at the chain-reduction layer relative to named cryptographic or probabilistic axioms. This convention follows EasyCrypt and CryptoVerif: chain-level reductions are kernel-checked relative to security assumptions; the assumptions themselves are not internally reducible.

shor_breaks_classical_under_crqc Conditioned on the existence of a CRQC capable of running Shor's algorithm [4] at the relevant key size, any classical signature scheme $\Sigma' \in \{\text{ECDSA}, \text{RSA-PSS}, \text{EdDSA}$ admits forgery with probability $1 - \text{negl}'$ where negl' is dominated by the classical key length.

aes_256_gcm_ind_cpa AES-256-GCM is IND-CPA-secure under the standard model: for any polynomial-time adversary $\mathcal{A}$ lacking the key, the IND-CPA distinguishing advantage is negligible in the AES security parameter $\lambda = 256$.

How to verify the kernel check. The Lean repository is public under the Paraxiom organization. Reproducing the kernel check takes one command after `elan` is installed:

```
git clone https://github.com/Paraxiom/identity-architecture-lean.git
cd identity-architecture-lean
git checkout e3806ac        # commit pinned to this treatise version
lake update                 # fetches Mathlib v4.27.0 (one-time)
lake build                  # ~5–8 min cold; expect 0 sorries, 0 errors
```

A representative kernel-checked proof is reproduced verbatim in Appendix D.

3. Layer theory

We name and motivate the six accretion layers, then prove their independence and ordering properties.

3.1. The six layers

L_0: **Substrate.** The material continuity — body, hardware, weights blob. Necessary precondition; identity-bearing in the sense that its destruction ends the locus (subject to Theorem 2.6).

L_1: **Narrative.** The locus's own claim of continuity: "I am the same entity that did X yesterday." Implemented variously as autobiographical memory, persistent state on disk, context window, institutional documentation.

L_2: **Relational.** Other accountability loci attest that $\mathcal{I}$ has continuity. Family, peers, federation members, protocol counterparties.

L_3: **Cryptographic.** A keypair (PQ-secured under the model assumptions of Section 9). The first layer that scales without prior relationship.

L_4: **Civic.** A jurisdictional attestation: birth certificate, passport, KYC bundle, corporate registration.

L_5: **Behavioral.** The cumulative track record of actions taken — votes, decisions, signed attestations, transferred resources, resolved disputes. Aggregates of L_3 and L_2 over time.

3.2. Diagram of accretion

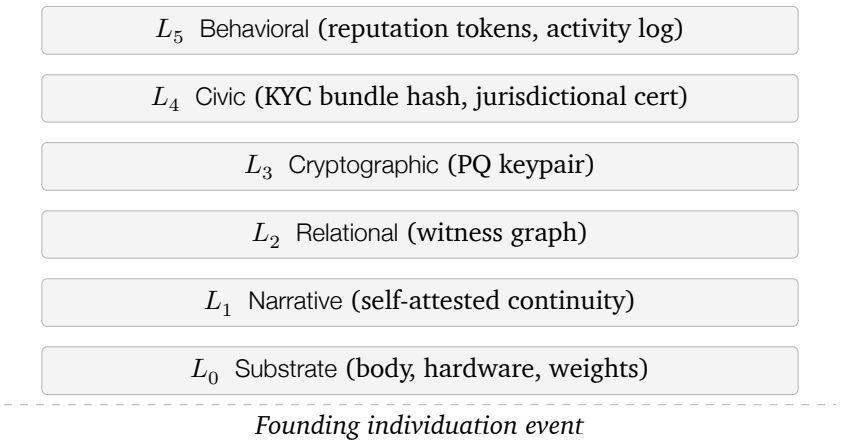

Figure 1: The six accretion layers above the founding individuation event. Each layer can fail independently; combined failure of $k \geq 4$ layers causes identity dissolution.

3.3. Layer composition is non-commutative in time

Proposition 3.1 (Layer accretion order is not strict)**.** *The temporal order in which layers accrete is not fixed. An identity may acquire L_3 before L_4 (a private key before legal recognition), L_2 before L_1 (others recognize an organism before the organism develops self-narrative), or L_5 before L_3 (a reputation accumulated under pseudonym, later bound to a key).*

Theorem 3.1 is consequential for design: the identity registry must permit any layer to be backfilled, not enforce a canonical ordering.

4. Founding events across substrates

We specialize the founding-event definition to the four canonical substrate types.

4.1. Biological founding

A human identity begins not at conception (continuous chemical process) nor at issuance of the birth certificate (administrative echo), but at *first individuation*: the moment after birth at which the organism's state is distinguishable from its parents'. Witnesses (the birthing parent and attending clinicians) populate the initial relational graph $\mathcal{R}$.

4.2. Machine founding

A host's identity begins at first boot: a host-unique identifier is written to persistent storage, entropy is harvested, and initial cryptographic material is established (TPM-derived keys, and the platform's host keypair — SSH host keys on Linux, the System UUID plus Keychain enrollment on macOS, AIKs plus the MachineGuid on Windows). The precise storage locations vary by platform; we treat them per-OS in Section 5.

4.3. Container founding

A container's identity begins at `docker run` (or equivalent runc/containerd invocation). The image hash supplies L_0 cryptographically; the container ID supplies L_1 as the self-narrative anchor.

4.4. LLM founding

An LLM substrate's identity begins at *weights initialization*: two checkpoints trained from different random seeds on identical data are distinct substrate identities, even when statistically indistinguishable in behavior. An LLM *instance* identity, by contrast, begins at *first inference call* on a deployed endpoint, when the deployment ID is assigned. We elaborate the substrate–instance distinction in Section 7.

4.5. Physical-good founding

A physical object — a luxury watch, a bottle of wine, a passport book, a banknote, a sensor module, a vial of pharmaceutical, a piece of art — bears identity as soon as it is individuated from a production batch. The founding event is one of: *manufacture* (a serial number assigned and a tamper-resistant identifier bound: NFC tag, RFID, holographic seal, laser engraving, Physical Unclonable Function (PUF) measurement), *mint* (for coins, banknotes, and minted collectibles), *harvest* (for agricultural goods: a coffee lot, a wine vintage, a fishing catch), or *packaging* (when individuation occurs at distribution rather than production — e.g. a sealed unit-of-sale).

A good is *passively identity-bearing*: it does not initiate actions itself, but the substrate cryptographically anchors a chain of custody. The L_3 key associated with the good is typically the manufacturer's signature over the founding artifact (the chip's attested public key, the seal's hash); subsequent L_5 accretions record ownership transfers, authentication events at retail, repair records, and verifications by relying parties. The accountability chain runs from the manufacturer through every custodian to the present holder; broken-chain detection (a counterfeit) corresponds to verification failure of the chain.

Production realizations as of May 2026 include LVMH's Aura blockchain (luxury goods, multi-brand, launched 2021), Arianee (luxury watches and jewellery, 2018), VeChain (food-trace, automotive parts, pharmaceutical cold-chain), IBM Food Trust (supply chain), Origin Protocol (collectibles), and the ERC-6551 token-bound-account ecosystem in which an individual NFT acts as both the identity record and a chain-native account for the physical object it represents. We return to the integration with these systems in Section 10 § 10.3.

4.6. Unifying schema

Domain	Substrate $\mathcal{S}$	Founding $\mathcal{F}$	First L_1/L_2 accretion
Human	Body	Birth	Birthing parent, attending clinicians
Machine	Hardware	First boot	Host identifier, first cryptographic m
Container	Image	Runtime instantiation	Container ID, mount state
Process	Code binary	First invocation	PID, parent, env
LLM substrate	Architecture + dataset	Weights init	Checkpoint hash, training log
LLM instance	Weights	First inference	Deployment ID, context
Physical good	Object	Manufacture / mint / harvest	NFC or RFID chip ID, holographic se

Table 2: Founding events by substrate domain.

5. Machine identity storage

The model is concrete: machine identity lives in named files, registers, and hardware tokens at known locations. Understanding the *permanence tier* of each storage location is the engineering

prerequisite for trusting the resulting identity claim.

5.1. The four-tier hierarchy

Definition 5.1 (Permanence tier)**.** A storage location for an identity artifact has *permanence tier* $T \in \{0, 1, 2, 3\}$ defined as follows. Tier 0 identifiers are *manufacturer-fused*: present at production and unchangeable without replacing hardware. Tier 1 identifiers are *installation-rooted*: established at first boot or first install, persistent across restarts, lost on reinstall. Tier 2 identifiers are *operationally configured*: persistent across restarts but routinely changeable by administrators. Tier 3 identifiers are *runtime-ephemeral*: regenerated each session.

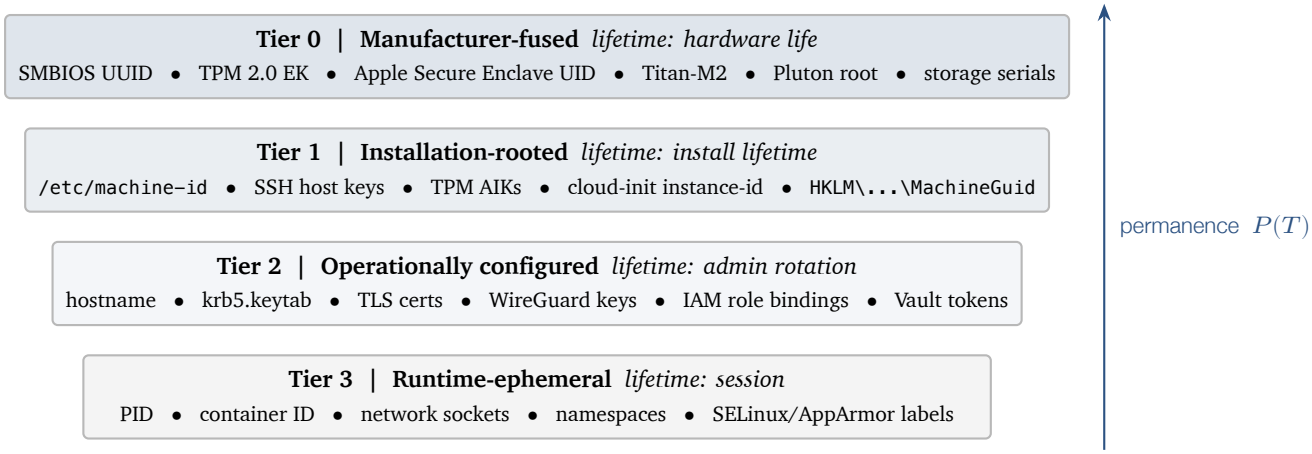

Figure 2: Permanence tier hierarchy. Theorem 5.2 states $P(0) \geq P(1) \geq P(2) \geq P(3)$. The post-quantum identity stack should select the lowest-tier identifier available when registering an agent's `instance_anchor` (Theorem 5.3).

5.2. Tier 0 (manufacturer-fused)

- **SMBIOS/DMI UUID**: burned at manufacture, queryable via `dmidecode -s system-uuid`. Resides in firmware tables, not on disk.
- **TPM 2.0 Endorsement Key (EK)**: RSA or ECC keypair burned into the TPM at manufacture, certified by the manufacturer's CA. Root of trust for hardware attestation.
- **Apple Secure Enclave UID, Google Titan-M2 keys, Microsoft Pluton root keys**: per-chip UIDs fused at manufacture, participating in cryptographic operations without exfiltration.
- **Storage device serials**: ATA/NVMe serials per physical disk, surviving reformat.
- **Network MAC addresses**: set per NIC at manufacture but software-spoofable, hence straddling Tier 0 and Tier 3.

5.3. Tier 1 (installation-rooted)

- **`/etc/machine-id`**: 128-bit identifier generated by `systemd-machine-id-setup` on first boot. The `systemd` documentation explicitly states that this ID "should be considered confidential and must not be exposed in untrusted environments," and that downstream applications must apply `sd_id128_get_machine_app_specific()` to derive per-application identifiers rather than using the raw value.
- **`/etc/ssh/ssh_host_{rsa,ecdsa,ed25519}_key`**: generated at first SSH daemon configuration. These are the host's first cryptographic identity artifact, anchoring SSH trust-on-first-use and SSHFP DNS records.
- **TPM-owned derived keys**: Storage Root Key, Attestation Identity Keys, platform-bound passkey credentials — generated at enrollment but cryptographically descended from the Tier-0 EK.
- **Cloud-init instance ID**: at `/var/lib/cloud/data/ instance-id`, mirroring the cloud provider's instance handle.

5.4. Tier 2 (operationally configured)

- **`/etc/hostname`**: human-readable. Conventional primary identifier but cryptographically meaningless on its own.
- **Kerberos host principal keys** (`/etc/krb5.keytab`); TLS server certificates; WireGuard keypairs; cloud IAM role bindings.

5.5. Tier 3 (runtime-ephemeral)

- PID, container ID, network state, in-kernel `struct cred`, namespaces, SELinux/AppArmor labels.

5.6. Tier monotonicity theorem

Theorem 5.2 (Tier monotonicity)**.** *Let $P(T)$ denote the unconditional probability that an identifier of permanence tier T survives the time interval $[t_0, t_0 + \Delta]$ for Δ on the order of a host's installation lifetime (∼1–10 years). Then*

$$P(0) \geq P(1) \geq P(2) \geq P(3),$$

with strict inequality $P(0) > P(3)$.

Proof. By Theorem 5.1 and operational invariants. Tier 0 identifiers are unchangeable without hardware replacement; the probability of hardware replacement over the operational lifetime defines $1 - P(0)$. Tier 1 identifiers are lost on reinstall, with reinstall rate strictly greater than hardware-replacement rate; thus $P(1) \leq P(0)$. Tier 2 identifiers are changed administratively at

rates strictly greater than reinstall rate (hostnames, certificates, IAM bindings rotate routinely); $P(2) \leq P(1)$. Tier 3 identifiers are lost on reboot or session end, occurring at rates dramatically greater than administrative configuration rotation; $P(3) \leq P(2)$ with strict inequality typical. Strict inequality $P(0) > P(3)$ follows from the typical many-orders-of-magnitude separation between hardware lifetime and process lifetime. ■

Corollary 5.3 (Substrate identifier selection)**.** *Given a choice of substrate identifiers available at registration time, the post-quantum identity stack should select the* lowest-numbered tier available *for the substrate identity field. For an agent running on bare metal with TPM, this is the TPM-quoted EK identifier; on virtualized hardware, the SMBIOS UUID; on containers without TPM passthrough, the container image hash plus container ID.*

Theorem 5.3 directly informs the `instance_anchor` field of the agent-identity record:

$$\texttt{instance\_anchor} := \mathsf{H}_{\text{Blake2-256}}(\text{tier}_{\min} \,\|\, \text{agent\_kind} \,\|\, t_{\text{spawn}}),$$

mixing the lowest-tier available identifier with runtime context to produce a stable yet instance-specific anchor.

5.7. Platform-specific tier locations

Tier	Linux	macOS	Windows
0	SMBIOS UUID; TPM EK; SoC UID	`IOPlatformUUID`; Secure Enclave UID	SMBIOS UUID; Pluton root
1	`/etc/machine-id`; SSH host keys; TPM AIKs	System UUID; Recovery key; Keychain enrollment	`HKLM\...\MachineGuid`; TPM AIKs
2	Hostname; krb5 keytab; TLS certs; WireGuard keys	Hostname; user keychain certs; configuration profiles	Computer SID; domain bindings
3	PID; container ID; namespaces	PID; sandbox ID	PID; session token

Table 3: Permanence tiers across major operating systems.

6. Agent identity

6.1. Three forms of agency

A *process* is the weakest agency: a running execution context with PID, parent, and loaded code. By default it carries no L_3 cryptographic claim; it inherits trust from its parent.

A *container* adds substrate cryptographic anchor by way of the image hash, which itself is signed by its publisher.

We distinguish agency from *passive identity-bearing*: a physical good (Section 4.5) bears identity but initiates no actions. Goods participate in accountability chains as loci attested-about, not as loci that attest. Their L_3 key is held by the manufacturer or current custodian, not by the good itself.

A *true agent*, in our sense, is a program acting on behalf of a principal. It requires:

1. Its own cryptographic identity (L_3): a Falcon-512 or SPHINCS+ keypair, generated at first registration.
2. A delegation chain from the principal (L_2): a parent identity in $\mathcal{R}$ tracing to an L_4-anchored or reputation-bonded L_3 identity.
3. Scope bounds: the set of capabilities the agent is authorized for.
4. Reputation (L_5): an accumulated track record.
5. Revocability: the principal must be able to revoke without cooperation from the agent.

6.2. The stratum lattice

We define a partial order $\prec$ on identity loci. Write $\mathcal{I}_1 \prec \mathcal{I}_2$ iff $\mathcal{I}_1$ appears as a parent identity for $\mathcal{I}_2$ in $\mathcal{R}$.

Definition 6.1 (Stratification)**.** The *stratification* of an identity space partitions loci into five strata:

S1 Person KYC-rooted human identity. Root of the $\prec$ order.

S2 Scoped Person $\times$ tenant $\times$ role. Parent: an S1.

S3 Vault Encrypted credential commitments. Held by an S1, S2, or organization S5; not itself an identity-bearing entity but a record attached to one.

S4 Agent instance Running agent. Parent: any of S1, S2, or S5.

S5 Cross-tenant Organizational aggregate identity. Parent: a multi-signature of S1 founders or jurisdictional authority.

6.3. Stratum well-foundedness

Theorem 6.2 (Stratum well-foundedness)**.** *The relation $\prec$ restricted to S2, S4, S5 identities induces a well-founded ordering: every non-S1 identity has an S1 ancestor in finitely many $\prec$-steps.*

Proof. Assume the chain registration policy of Section 11: every S2/S4 identity registration extrinsic requires a non-null `parent_identity` pointer, and the registration is chain-validated against the existence of the parent. Every S5 identity registration requires either a multi-signature of S1 founders or a jurisdictional L_4 claim, both of which terminate at S1 or at jurisdictional authority (treated as an S1 surrogate). Suppose for contradiction an infinite descending chain $\mathcal{I}_0 \succ \mathcal{I}_1 \succ \mathcal{I}_2 \succ \ldots$. Each $\mathcal{I}_i$ is recorded on chain with a block height h_i, and the parent's

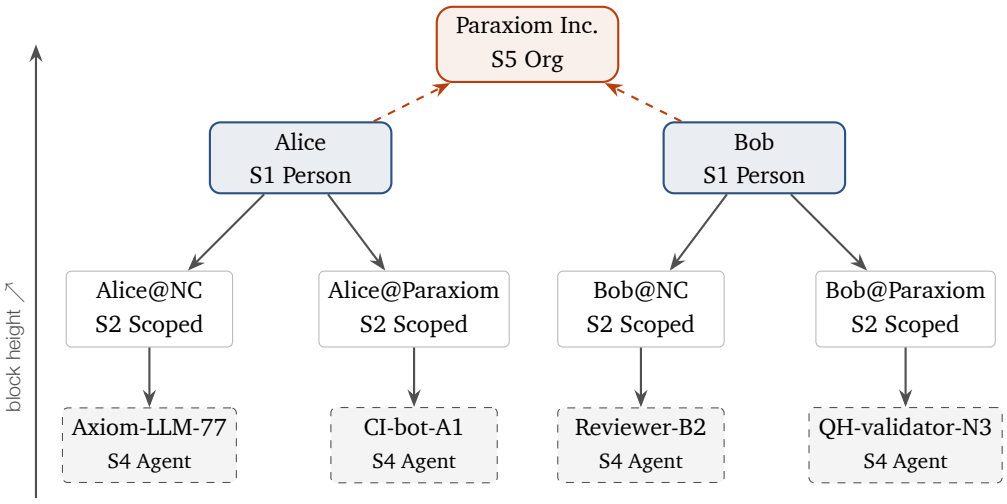

Figure 3: The stratum lattice. Every S2 / S4 identity registers with a non-null `parent_identity` field, terminating in finite $\prec$-steps at an S1 root (Theorem 6.2). S5 organizational identities are rooted in a multi-signature of S1 founders (dashed arrows). Block heights are strictly monotone parent→child, precluding cycles.

registration precedes the child's, so $h_0 > h_1 > h_2 > \ldots$. This is an infinite descending chain of natural numbers — impossible. Hence finite-length termination at S1 (or an S1 surrogate) is guaranteed. ■

Theorem 6.2 is the formal restatement of "no accountability chain, no identity" (Section 2): every action by an S2/S4 identity traces in finite steps to a human or jurisdictional accountability locus.

6.4. Liability follows the chain

Corollary 6.3 (Liability attribution). *For any action α taken by an identity $\mathcal{I}$, liability attaches to: (i) $\mathcal{I}$ itself if it carries a stake or bond; (ii) the parent identity in $\prec$ if $\mathcal{I}$ is S2 or S4 and unable to satisfy the liability; (iii) recursively to the root S1 identity, by Theorem 6.2.*

7. LLM identity

LLMs are the test case that breaks naive substrate-equals-identity reasoning: weights are bit-perfectly copyable.

7.1. Substrate vs. instance

Definition 7.1 (LLM substrate). An *LLM substrate* is a tuple $(\mathbf{W}, \tau, A)$ where $\mathbf{W}$ is a weights blob, τ is a tokenizer/architecture specification, and A is a training-run attestation. We write $\mathsf{H}(\mathbf{W})$ for the canonical substrate hash.

Definition 7.2 (LLM instance)**.** An *LLM instance* is a tuple $(\mathbf{W}_{\text{ref}}, d, c)$ where $\mathbf{W}_{\text{ref}}$ is a reference to a substrate hash, d is a deployment identifier (persistent across restarts), and c is the conversational state.

7.2. Instance non-identity theorem

Theorem 7.3 (LLM instance non-identity)**.** *Two LLM instances $\mathcal{I}_1 = (\mathbf{W}, d_1, c_1)$ and $\mathcal{I}_2 = (\mathbf{W}, d_2, c_2)$ with identical substrate reference but $d_1 \neq d_2$ are distinct identity loci.*

Proof. By Theorem 2.3 an identity locus is determined by its founding event $\mathcal{F}$. By Theorem 7.2 the deployment identifier d is established at first inference call, hence participates in $\mathcal{F}$. Since $d_1 \neq d_2$, the founding events $\mathcal{F}_1 \neq \mathcal{F}_2$, hence by Theorem 2.8 the identity loci are distinct. ■

7.3. LLM tier model

The four-tier permanence hierarchy of Section 5 maps to LLM identity:

Tier	LLM storage location
0	Weights hash; training-run Merkle root; architecture spec hash; TEE attestation (Intel TDX / AMD SEV-SNP / NVIDIA H100 confidential computing / Apple Private Cloud Compute) when applicable
1	Deployment ID; LoRA adapter hash; system-prompt hash; tool/function-call registry
2	API authentication; rate-limit/quota config; routing config; vector-DB or memory-store binding
3	Conversation thread ID; context window; tool-call sequence; KV cache

Table 4: LLM identity by permanence tier (parallel to Table 3).

7.4. The substrate attestation pallet

We introduce a candidate runtime pallet, `pallet_substrate_attestation`, that records LLM substrate provenance on chain. The extrinsic shape:

```
pub fn attest_substrate(
    origin: OriginFor<T>,
    weights_hash: H256,
    training_run_merkle: H256,
    arch_spec_hash: H256,
    publisher_falcon_sig: BoundedVec<u8, ConstU32<2048>>,
    ipfs_cid_card: BoundedVec<u8, ConstU32<128>>,
) -> DispatchResult;
```

The pallet binds a weights hash to a training organization's Falcon-512 public key. An LLM instance, on registration as an S4 agent, refers to a published substrate attestation. Verifiers can confirm: the running agent claims the substrate that organization O publicly attested to. The IPFS CID points to the model card and training-run log; the on-chain record holds only hashes and the publisher signature.

8. Governance levels and identity participation

A treatment of identity that stops at *individual, organization, swarm, protocol* misses the most consequential structural fact: a single identity participates in many distinct governance levels simultaneously, each with its own legitimacy source, scope, and reputation ledger. The identity is one; the governance memberships are many. This section formalizes the multi-level participation structure and proves the key non-bleeding property that makes the architecture work.

8.1. The governance-level taxonomy

Definition 8.1 (Governance level taxonomy)**.** The governance-level set $\mathbb{G}^{\text{lvl}} = \{G_0, G_1, \dots, G_7\}$ enumerates eight governance scopes:

G_0 ***Self*** Sovereign control over one's own state, body, key material, vault. Constituency size: 1.

G_1 ***Domestic*** Household / family / intimate group. Shared finances, dependents, shared assets.

G_2 ***Project / committee*** Working team, ad-hoc body, time-bound initiative.

G_3 ***Organizational*** Companies, NGOs, formal chartered entities.

G_4 ***Federation*** Voluntary inter-organizational cooperation.

G_5 ***Civic*** Municipal, provincial/state, national governance.

G_6 ***Supranational*** Treaty bodies, alliances spanning sovereigns.

G_7 ***Protocol*** Decentralized on-chain governance, DAOs, protocol councils.

The levels are not strictly hierarchical: G_3 does not necessarily inherit from G_5; G_7 may operate cross-jurisdictionally.

8.2. Identity participates in many levels at once

A single identity locus typically participates in several bodies across multiple G-levels simultaneously. For example, a person $\mathcal{I}$ may be the sole member of his G_0 self-sovereign body; participate in a G_1 household; serve on a G_2 project committee; hold an officer role in a G_3 corporation; be a member of a G_4 industry federation; be a citizen of G_5 civic bodies at multiple geographic granularities; be transitively bound by his G_5 jurisdiction's G_6 treaty obligations; and hold G_7 protocol identities across multiple DAOs. The accountability chain does not fork into many identities; it forks into one identity with many scoped participations.

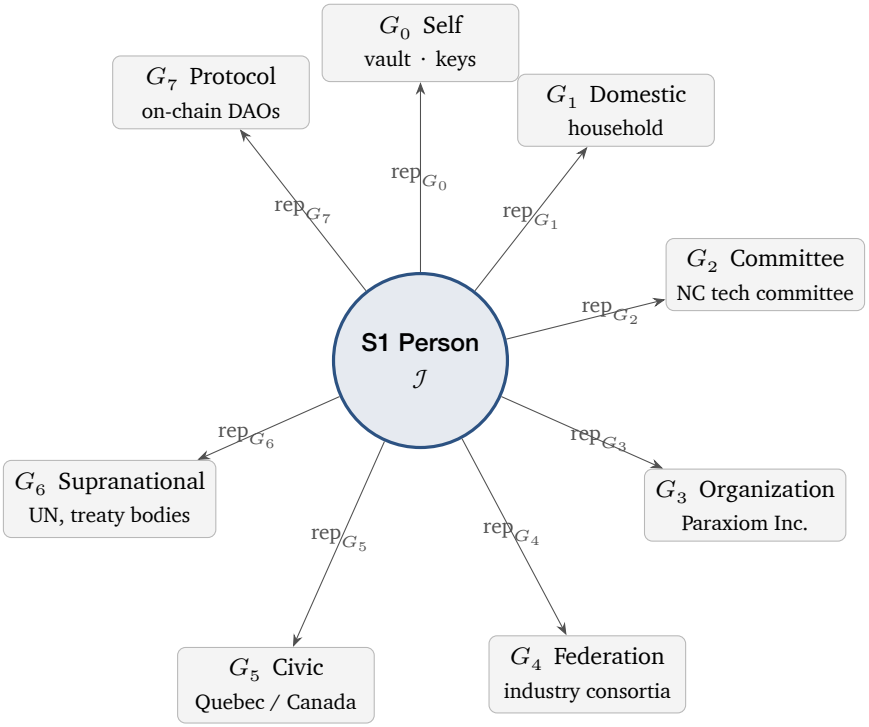

Figure 4: Multi-level participation. A single S1 identity locus participates in up to eight governance bodies at distinct G-levels simultaneously. Each participation is a separate membership record with its own scoped reputation ledger $\mathrm{rep} : \mathbb{I} \times \mathbb{B} \to \mathbb{N}$. Theorem 8.6 states that reputation does not flow between these bodies without explicit cross-body endorsement.

8.3. Governance bodies as first-class objects

Definition 8.2 (Governance body)**.** A *governance body* is a tuple

$$B = (b, \ell, \pi_B, \chi, \sigma_\pi, \mathcal{M}_B)$$

where:

- $b \in \mathbb{N}$ is the body identifier
- $\ell \in \mathbb{G}^{\mathrm{lvl}}$ is the governance level
- π_B is the parent body, or $\perp$ if sovereign
- $\chi \subseteq \mathbb{S}^{\mathrm{scope}}$ is the *chartered scope* — the decision domains the body is authorized to act upon, drawn from a primitive scope-set $\mathbb{S}^{\mathrm{scope}}$
- σ_π is the parent body's PQ signature over (b, ℓ, χ), or a founding-event attestation for sovereign bodies
- $\mathcal{M}_B \subseteq \mathbb{I}$ is the membership

We write $\mathbb{B}$ for the set of all governance bodies.

Definition 8.3 (Participation relation)**.** The *participation relation* is $P \subseteq \mathbb{I} \times \mathbb{B}$ where $(\mathcal{I}, B) \in P$ iff $\mathcal{I} \in \mathcal{M}_B$ at the inquiry time. We write $P(\mathcal{I}) := \{B : (\mathcal{I}, B) \in P\}$ for the set of bodies in which $\mathcal{I}$ participates.

8.4. Chartered scope bounding

Theorem 8.4 (Chartered scope bounding)**.** *For any governance body B with parent π_B:*

$$\chi_B \subseteq \chi_{\pi_B}.$$

That is, a child body's chartered scope is a subset of its parent's chartered scope. Sovereign bodies bound their own scope by founding-event attestation.

Proof. The on-chain registration extrinsic for a body requires the parent's PQ signature σ_π over the proposed χ. The runtime validator accepts the registration only after confirming $\chi \subseteq \chi_{\pi_B}$ by direct set comparison. If $\chi \not\subseteq \chi_{\pi_B}$, the extrinsic is rejected. Sovereign bodies do not have a parent, but their χ is bounded by the founding-event attestation, verifiable by any verifier. The bound holds in both cases. ■

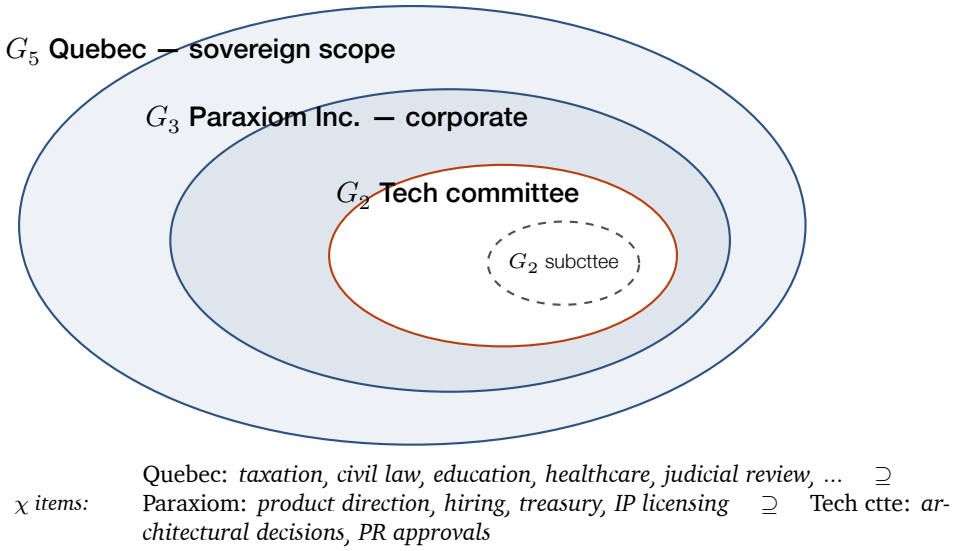

Figure 5: Chartered scope bounding (Theorem 8.4). A child body's chartered scope χ is a strict subset of its parent's. A G_2 committee within a G_3 corporation may only decide what the corporation chartered to it; the corporation, in turn, may only act within scopes available to it under its G_5 jurisdiction. The chain rejects `register_body` extrinsics that violate the subset relation.

Theorem 8.4 is the formal restatement of the principle that a child cannot claim authority the parent does not have. A G_2 committee within a G_3 organization may make decisions only within the sub-scope the organization delegated; a G_3 corporation operating under G_5 jurisdiction may not exercise powers reserved to the G_5 sovereign.

8.5. Multi-level participation theorem

Theorem 8.5 (Multi-level participation independence)**.** *Let $\mathcal{J} \in \mathbb{I}$ with $P(\mathcal{J}) = \{B_1, \ldots, B_n\}$. If the chartered scopes are pairwise disjoint, $\chi_{B_i} \cap \chi_{B_j} = \emptyset$ for all $i \neq j$, then $\mathcal{J}$'s authorization to act in any B_i is independent of its authorizations in $B_{j \neq i}$. The action-authorization predicate* $\text{auth}(\mathcal{J}, B_i, \alpha)$ *depends only on $\mathcal{J}$'s membership and reputation in B_i.*

Proof. The action-authorization predicate is

$$\text{auth}(\mathcal{J}, B, \alpha) = \text{member}(\mathcal{J}, B) \ \wedge \ (\alpha \in \chi_B) \ \wedge \ (\text{rep}(\mathcal{J}, B) \geq \theta_{B,\alpha}),$$

where $\theta_{B,\alpha}$ is the per-body, per-action minimum reputation threshold. The membership predicate queries P for the specific $(\mathcal{J}, B)$ pair; the scope check uses B's chartered scope only; the reputation check queries the per-body reputation table. None of these depend on $\mathcal{J}$'s membership or reputation in any other body. Hence the authorization is independent across bodies whose scopes are disjoint. ■

Theorem 8.5 formalizes the design claim that a person's reputation in a G_2 committee does not automatically grant authority in a G_7 protocol DAO. The authorization machinery is local to each body's scope.

8.6. Reputation non-transitivity

Theorem 8.6 (Reputation non-transitivity)**.** *For governance bodies $B_a \neq B_b$ and no explicit cross-body endorsement, the reputation function* $\text{rep} : \mathbb{I} \times \mathbb{B} \to \mathbb{N}$ *satisfies*

$$\text{rep}(\mathcal{J}, B_a) \ \not\Rightarrow \ \text{rep}(\mathcal{J}, B_b).$$

Cross-body recognition requires a third body B_c with chartered scope including both B_a and B_b to issue an `axiom-attestation` *of variant* `CrossBodyEndorsement` *linking the two.*

Proof. The reputation function is implemented as a per-body storage map $(\text{holder\_id}, \text{body\_id}, \text{token\_kind}) \to$ TokenRecord. Queries against $\text{rep}(\mathcal{J}, B_b)$ access only B_b-keyed entries; no key in this map references B_a. Hence $\text{rep}(\mathcal{J}, B_a)$ contributes nothing to $\text{rep}(\mathcal{J}, B_b)$. The cross-body endorsement extrinsic explicitly creates a B_b-keyed entry derived from a B_a-keyed entry, gated by an authorizing signature from a body whose chartered scope contains both. Absent such an endorsement, the two reputations are independent. ■

Remark 8.7. Theorem 8.6 structurally defeats a class of Sybil attacks: an adversary cannot accumulate cheap reputation in an under-defended body and then exercise it in a high-stakes body. Each governance body bears responsibility for the integrity of its own reputation distribution.

8.7. Augmented democracy and multi-level structure

The *augmented democracy* architecture[1] described here — transparent multi-stakeholder governance for cities and enterprises — places governance bodies at multiple G-levels in interaction. The architecture is general; it admits a characteristic deployment pattern in which:

- A G_2 committee (e.g. a technical steering committee) makes product decisions that bind a G_3 operating organization.
- The G_3 organization, via its G_2 committee, ships decisions as software changes attested on-chain by a G_7 protocol body (QuantumHarmony) through an attestation pallet.
- Customer organizations are themselves G_3 bodies operating on the platform; their internal G_2 committees use the same governance-body machinery, scoped to the customer's tenant context (an S_2 stratum identity).
- All of the above are subject to G_5 jurisdictional law (e.g. a national or provincial registrar) via the G_3 corporation's `jurisdictional_anchor`, which links the body to its registering authority's L_4 claim.

The architecture is the canonical augmented-democracy substrate specified by this treatise. Subsequent deployments (municipal-governance pilots and enterprise compliance pilots) consume the same substrate without modification.

The chain holds all of these as first-class objects via `pallet-governance-bodies` (Appendix B). The verification ladder (question generation, PR review, deliberation) operates within each body's scope, returning a verdict only valid for that body's decision space.

9. Cryptographic substrate

9.1. The post-quantum imperative

A classical signature is a time bomb. Suppose at time t a signature $\sigma = \mathsf{Sign}_{sk}(c)$ is produced under classical ECDSA. At time $t' > t$, a CRQC becomes available. Any party with access to the CRQC can compute a forgery $\sigma^\dagger$ on an arbitrary message $c^\dagger$ that verifies under the same public key. From the verifier's perspective at t', σ and $\sigma^\dagger$ are indistinguishable: classical EUF-CMA security collapses retroactively.

Unlike encryption (where session keys can be rotated forward), *signatures bound to historical claims cannot be rotated*: once a signature is part of the accretion chain, it is part of the record.

[1] The term *augmented democracy* was introduced by the present author in August 2017 [20], predating the more widely known TED talk popularization of a similar phrase (Hidalgo, September 2018) by approximately thirteen months. The 2017 work develops the machine-based societal model for curbing citizen cynicism that this treatise's governance layer realizes on chain.

9.2. Cryptographic survival theorem

Let Σ_{PQ} denote the set of NIST-standardized post-quantum signature schemes ($\Sigma_{\text{PQ}} = \{\text{ML-DSA}, \text{SLH-DSA}, \text{FN-DS}$ as of 2024–2026), and let Σ_{cl} denote classical schemes (ECDSA, RSA-PSS, EdDSA).

Theorem 9.1 (Cryptographic survival)**.** *Let $\mathcal{I}$ be an identity locus with accretion chain $\mathcal{A}$ in which every L_3 claim uses some scheme $\Sigma \in \Sigma_{\text{PQ}}$. Let T_{CRQC} denote the (uncertain) time at which a sufficiently-capable CRQC becomes available. Then for any verifier inquiring at any time $t > T_{\text{CRQC}}$, the EUF-CMA-forgery probability of $\mathcal{I}$'s accretion chain is*

$$\Pr[\textit{forgery}] \leq \text{negl}(\lambda),$$

where λ is the post-quantum security parameter of Σ (typically ≥ 128).

Conversely, if any L_3 claim uses $\Sigma' \in \Sigma_{\text{cl}}$, then

$$\Pr[\textit{forgery}] \geq 1 - \text{negl}'(\lambda'),$$

where negl$'$ *characterizes the residual difficulty for the CRQC, and the bound is dominated by the classical key length, not the post-quantum security parameter.*

Proof sketch. The forward direction follows from the EUF-CMA security of $\Sigma \in \Sigma_{\text{PQ}}$ under the post-quantum adversarial model assumed in NIST FIPS 203–205 [1, 2, 3]. The reverse direction follows from Shor's algorithm [4], which solves discrete-log in $\mathbb{Z}_p^*$ and elliptic-curve groups in polynomial quantum time, breaking ECDSA and RSA-based signatures with overwhelming probability once the CRQC operates above the corresponding key-size threshold. ■

Corollary 9.2 (Engineering implication)**.** *For any accretion chain intended to remain verifiable past T_{CRQC}, every L_3 claim must be signed under $\Sigma \in \Sigma_{\text{PQ}}$* at the time the claim is made. *Retrofitting PQ signatures onto classical-signed historical claims is impossible: the historical signatures already exist and are already forgeable.*

Theorem 9.2 is the operational claim that QuantumHarmony implements SPHINCS+ (SLH-DSA, FIPS 205) and Falcon-512 (FN-DSA, FIPS draft 206) as *primary* signature schemes, not as opt-in upgrades. Every identity-bearing transaction is PQ-signed at issuance.

9.3. Signature size accommodation

PQ signatures are larger than classical signatures. SPHINCS+-256f-simple is 49,856 bytes; Ed25519 is 64 bytes. The Paraxiom fork of `polkadot-sdk` raises `MAX_SIGNATURE_SIZE` to 50,000 and `parity-scale-codec`'s `INITIAL_PREALLOCATION` from 16 KiB to 256 KiB to accommodate runtime metadata containing PQ signatures [8]. The trade-off (chain storage growth) is intentional: chain history must persist; signature size is the cost of identity survival.

10. Current digital identity landscape (May 2026)

The formal model contacts deployed reality. We summarize the state of digital identity at the time of writing, drawing on regulator filings, standards publications, and operator disclosures. Detailed citations are in the bibliography.

10.1. Civic identity at scale

Estonia. ID-card, Mobile-ID, Smart-ID. Classical cryptography (RSA, ECDSA-P384). Cybernetica contracted in 2026 to author the national PQC roadmap as part of the EU joint PQC plan. KSI-blockchain audit log is hash-based and quantum-resilient; identity-binding primitives are not.

India — Aadhaar. 1.4 billion enrolled. Authentication failure rate $\sim$6.5% per attempt (Policy Circle, June 2025); roughly 20.3 million failed attempts per month. 30 million ration cards cancelled due to seeding mismatches. 815 million records exposed in 2023; 29,000 biometric-cloning fraud incidents in 2024. Architecturally a centralized database lookup, not a cryptographic credential system. The canonical anti-pattern.

EU — eIDAS 2.0 / EUDI Wallet. Regulation EU 2024/1183 in force May 2024. Implementing acts December 2024. Hard deadline: 24 December 2026, every Member State must offer at least one EUDI Wallet; mandatory acceptance by relying parties December 2027. Wire protocols: OpenID4VP 1.0 Final, OpenID4VCI 1.0 Final, HAIP 1.0. Credential formats: SD-JWT VC and ISO mdoc. PQC on a separate clock: high-risk migration by 2030, full by 2035. Identity-wallet PKI explicitly flagged as longest-lead-time transition.

UK — GOV.UK One Login. 11 million users. Becomes exclusive access to central government services by end-2027. Currently lacks certification under the UK Digital Identity & Attributes Trust Framework (DIATF) due to a supplier accreditation lapse — the national IDP operating uncertified against the country's own framework.

US — mDL. No federal ID. State-level under AAMVA Implementation Guidelines v1.4–1.5. As of January 2026, 21 states plus Puerto Rico operate mDL programs accepted at TSA. ISO mdoc with COSE-signed claims. The only US system with cryptographic primitives at meaningful scale.

Canada. Verified.me effectively defunct after SecureKey's 2022 sale. De-facto trust layer is DIACC's Pan-Canadian Trust Framework (PCTF). No federal wallet equivalent to EUDI.

Singapore, Brazil, Nigeria. SingPass ($\sim$5M residents); gov.br ($\sim$150M users with centralizing biometric service launched 2025); Nigeria NIN with biometric General Multipurpose Card from January 2025. All architecturally centralized database-lookup; growing scale, growing exclusion.

Worldcoin / World ID. Banned, ordered to delete data, or under active investigation in Colombia, Brazil, Thailand, Philippines, Hong Kong, Spain, France, Portugal, Argentina, Kenya. Repeated regulatory finding: financial incentives ($\$10$–$\$50$ in WLD per iris scan) vitiate informed consent under GDPR-style biometric regimes. Not a deployable architecture.

10.2. Self-sovereign identity — what shipped

- W3C Verifiable Credentials Data Model 2.0 became a W3C Recommendation on 15 May 2025.
- W3C DIDs Core 1.1 reached Candidate Recommendation Snapshot on 5 March 2026; DID Resolution split into separate v0.3.
- OpenID4VP 1.0 Final and OpenID4VCI 1.0 Final published July 2025; HAIP 1.0 Final December 2025. Selected as eIDAS 2.0 ARF wire protocols.
- Hyperledger Aries archived April 2025; components migrated to the OpenWallet Foundation.
- `did:web` is the only DID method with material non-toy adoption (UN Transparency Protocol, Microsoft Entra Verified ID). `did:ion` lost momentum.

The deployed SSI surface is `did:web` plus OpenID4VP/VCI plus SD-JWT VC or ISO mdoc. Everything else remains research-track.

10.3. Blockchain-anchored identity

ENS retains the only material adoption (1.7M+ domains; ENSv2 mainnet February 2026). Sismo wound down late 2024. Spruce ID pivoted to government mDL integrations and now operates chain-agnostically. Proof-of-personhood remains unsolved with no production winner.

NFT-based identity primitives

A distinct lane within blockchain-anchored identity has emerged around NFT-based primitives that bind on-chain identity records to off-chain substrates — humans, organizations, agents, and crucially *physical goods* (Section 4.5). Four sub-lanes have material 2026 traction.

Soulbound tokens (SBTs). Proposed by Weyl, Ohlhaver, and Buterin in *Decentralized Society: Finding Web3's Soul* [21], SBTs are non-transferable on-chain credentials whose binding to a holder is structural rather than market-mediated. The design space overlaps with this treatise's earned-reputation tokens (Section 11) and with verifiable-credential commitments, but the published SBT corpus does not commit to a post-quantum signature surface and does not formalize either stratum well-foundedness (Theorem 6.2) or verification-time accountability (Theorem 11.3).

Dynamic NFTs. [22] An on-chain object whose state evolves through oracle inputs, time-based rules, or holder actions. The dynamic-NFT primitive is structurally adjacent to the identity-state hash of Theorem 2.14: both treat identity as an evolving on-chain commitment addressable by current state rather than by a fixed handle. Three differences are operational: (i) most dynamic-NFT implementations carry a single integer or string state, not a hypergraph state; (ii) the binding property of Theorem 2.15 is not made explicit in the dynamic-NFT corpus; (iii) the underlying signature surface remains classical (ECDSA on secp256k1) and offers no CRQC survival.

ERC-6551 token-bound accounts. [23] Promotes individual NFTs to first-class identity-bearing accounts with their own asset balance and signing capability. The mechanism is structurally adjacent to this treatise's S4 agent-instance stratum (Theorem 6.1): each token is an account, with the issuing NFT contract as a kind of parent. The chain semantics differ: ERC-6551 does not enforce the parent-identity chain that Theorem 6.2 requires (an account's controller can change without recording lineage), and it does not formalize a revocation or crypto-shredding pathway.

NFT-bound physical goods. A production-scale lane in which the NFT serves as the digital twin of a physical object, with authentication anchored by NFC, RFID, holographic seal, or PUF measurement. Representative 2026 deployments: *LVMH Aura* (multi-brand luxury, 6M items registered across LVMH maisons; launched 2021), *Arianee* (luxury watches and jewellery; founded 2018), *VeChain* (food-trace, automotive parts, pharmaceutical cold-chain), *IBM Food Trust* (supply-chain provenance across grower, processor, distributor, retailer), and *Origin Protocol* (collectible goods).

These systems realize physical-good identity in operation, mapping cleanly to the *Physical good* domain row of Table 1: the manufacturer's signed founding event is anchored on chain; subsequent transfers, repairs, and authentication events accrete to the chain-of-custody. Architecturally, the present treatise's primitives extend these lanes in three directions:

1. Post-quantum signature surface (SPHINCS+/Falcon-512) on the founding-event and custody-transfer signatures, which the classical-signature lane cannot match;
2. Formal accountability tracing (Theorem 6.2, Theorem 11.3) over the chain-of-custody, with finite-step termination at the manufacturer or jurisdictional registrar;
3. Hypergraph identity state (Theorem 2.12, Theorem 2.14) for goods participating in multi-party certifications (organic + fair-trade + provenance + jurisdictional origin), which the single-integer-state of mainstream dynamic-NFT implementations does not capture.

The NFT-based lane is in production but *classical-signature-only* as of May 2026; migration to a post-quantum signature surface is not on any major roadmap and is the lane this treatise opens.

10.4. Hardware-rooted identity

- WebAuthn / Passkeys / FIDO2: more than 1 billion users with at least one passkey activated; more than 15 billion accounts support passkeys. Apple/Google/Microsoft alignment

held through 2025; FIDO CTAP 2.2 advanced cross-platform portability. HID/FIDO 2025 enterprise survey reports 87% of enterprises actively deploying passkeys.

- DICE (Device Identifier Composition Engine, TCG specification): now the dominant root-of-trust pattern on constrained devices.
- Apple Secure Enclave, Google Titan-M2, Microsoft Pluton: all baseline shipping on consumer devices.
- Intel SGX: effectively abandoned for client-side use; deprecated on 11th-generation+ consumer CPUs. Server-side use shifted to Intel TDX and AMD SEV-SNP. ARM CCA Realms ships in Armv9.

10.5. Post-quantum cryptography status

- NIST FIPS 203 (ML-KEM), 204 (ML-DSA), 205 (SLH-DSA) published 13 August 2024. FIPS 206 (FN-DSA / Falcon) in draft.
- HQC announced March 2025 as second KEM (backup to ML-KEM).
- US OMB PQC migration guidance, required within one year of NIST standards, was overdue as of May 2026: still in draft.
- NSA CNSA 2.0: PQC required for National Security Systems by 2030; classical algorithms prohibited by 2035.
- EU joint PQC roadmap (June 2025): national strategies by end-2026; high-risk migration by 2030; full by 2035.
- **No major civic identity system runs PQC signatures in production as of May 2026.**

10.6. AI agent identity — the open lane

- Agentic AI Foundation (Linux Foundation) launched December 2025. Anthropic contributed MCP; OpenAI contributed AGENTS.md; Block contributed Goose.
- MCP authorization standardized OAuth 2.1 + PKCE in March 2025; the November 2025 release added a *server identity* primitive.
- Google A2A protocol (April 2025) uses workload-identity federation: short-lived cryptographically verifiable credentials.
- IETF Web Bot Auth draft (Cloudflare-backed) signs HTTP requests with private keys plus a published directory.
- GNAP (RFC 9635, October 2024): most plausible OAuth successor for delegated agent authorization; minimal vendor uptake.
- Model substrate attestation: *no agreed standard*. NVIDIA H100/B200, AMD SEV-SNP, Intel TDX supply hardware; the protocol layer is open.

The state of AI agent identity in 2026 is comparable to the state of SSI in 2018: many overlapping proposals, no production winner, vendors shipping ahead of standards.

10.7. Positioning fit for Paraxiom

Layer	Industry default 2026	Paraxiom choice	Lane
Wallet protocol	OID4VP/VCI + HAIP	Implement OID4VP-compatible verifier	Track
Credential format	SD-JWT VC + ISO mdoc	Support both for export	Track
DID method	`did:web` for interop	`did:web` + QH-anchored method	Track + extend
Signature crypto	RSA / ECDSA (classical)	SPHINCS+ / Falcon-512 (PQ)	**Lead**
Hardware root	Apple SE / Titan-M2 / Pluton	Same + DICE for edge	Track
Agent identity	MCP + OAuth 2.1 (early)	MCP + Falcon-signed + on-chain	**Lead**
Model attestation	None (open)	`pallet_substrate_attestation`	**Lead**
Trust framework	EU eIDAS / DIACC / state	Chain-native, bridge to civic	Sovereign

Table 5: Paraxiom positioning against the 2026 landscape. Three uncontested lanes: post-quantum civic-grade identity, agent cryptographic identity, model substrate attestation.

10.8. Positioning vs. W3C VC 2.0 and DIDs 1.1

The W3C identity stack as of May 2026 consists of *Verifiable Credentials Data Model 2.0* (Recommendation, 15 May 2025 [11]) and *Decentralized Identifiers v1.1* (Candidate Recommendation Snapshot, 5 March 2026 [12]), complemented by the OpenID Foundation's OpenID4VP / OpenID4VCI 1.0 Final wire protocols. Together these define point-in-time credential issuance, holder-mediated selective-disclosure presentation, and identifier resolution. They do *not* define a number of primitives this treatise contributes; we tabulate the gap explicitly in Table 6.

Reading. W3C 2025–2026 standardizes the wire format and resolution layer of digital identity: how a credential is encoded, how an identifier resolves to a document, how a holder presents a selected subset of claims to a verifier. These are presentation-time primitives. The present treatise contributes the *model* layer beneath them: what identity *is* as a formal object, how it accretes, how it traces, how it survives a CRQC event, and how it extends to substrates the W3C corpus has not yet named (LLM weights, agent instances, physical goods, hypergraph relations). The two stacks compose: a Paraxiom-chain identity can issue and present verifiable credentials in W3C-compliant formats; conversely, a W3C-issued credential can be committed in the credential-vault (Theorem 11.1) and presented under any of the privacy tiers of Theorem 12.2.

11. The Paraxiom architecture

We now describe the engineering realization.

11.1. Three-tier system

The Paraxiom identity stack is decomposed into three layers of storage with three distinct trust and performance profiles:

The chain is the source of truth for cryptographic claims. IPFS is the canonical bulk-payload store, content-addressed by CID. The Postgres+Redis mirror is rebuildable from chain plus IPFS without information loss (modulo unrecoverable plaintexts, which are protected by deletion of the master key per Theorem 11.1).

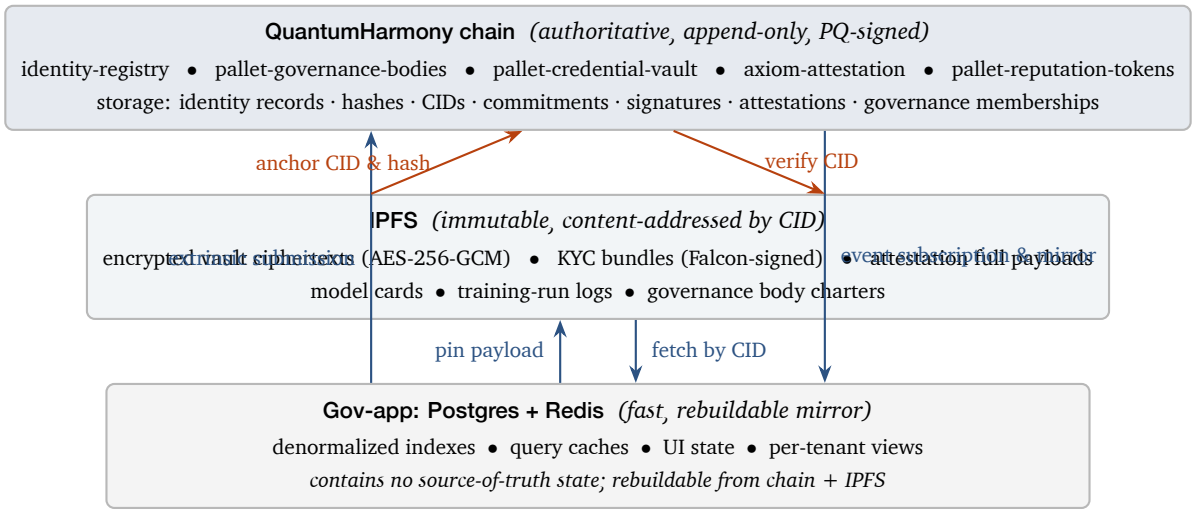

Figure 6: Three-tier identity architecture. The chain is the authoritative source of truth for cryptographic claims; IPFS holds bulk encrypted payloads addressed by content; Postgres+Redis is a fast, rebuildable query mirror. CCPA crypto-shredding (Theorem 11.1) is realized at the gov-app layer: deletion of the user's master key renders all IPFS-pinned ciphertexts permanently inaccessible without unpinning.

11.2. Identity strata

The five strata of Theorem 6.1 are realized on chain through the existing `identity-registry` pallet (extended) and a small number of companion pallets:

```
pub struct Identity<AccountId, BlockNumber> {
    pub id: u64,
    pub owner: AccountId,
    pub public_key: BoundedVec<u8, ConstU32<2048>>,
    pub role: IdentityRole,
    pub status: IdentityStatus,
    pub reputation_bps: u32,
    pub created_at_block: BlockNumber,
    pub created_by_tx_hash: H256,
    pub key_rotation_history: BoundedVec<KeyRotation, ConstU32<32>>,

    /// Stratum tag (S1, S2, S4, S5).
    pub stratum: Stratum,
```

```
    /// Parent identity for S2 and S4 (required by Theorem 6.2).
    pub parent_identity: Option<u64>,

    /// Tenant context for S2.
    pub tenant_id: Option<BoundedVec<u8, ConstU32<64>>>,

    /// Custody mode (SelfSovereign / NcCustodial / TenantCustodial).
    pub custody: CustodyMode,

    /// KYC bundle hash for S1 (Falcon-signed by KYC issuer).
    pub kyc_bundle_hash: Option<[u8; 32]>,

    /// Instance anchor for S4 agents (per Corollary 5.1).
    pub instance_anchor: Option<[u8; 32]>,
}
```

11.3. Companion pallets

`pallet-onboarding-commit` (extended): a generalized commit-reveal flow with bond and slashable dispute window. Used to bootstrap new S1 identities, agent S4 identities, and validators.

`axiom-attestation` (extended): on-chain Falcon-signed attestations with IPFS CIDs for full payloads. Extended with a `VerifierKind` enum to dispatch question-generation, PR-code-review, vault-commitment, persona-attestation, and existing Axiom-task variants.

`pallet-credential-vault` (new): on-chain commitments to off-chain envelope-encrypted credentials. Stores `(ciphertext_cid, ciphertext_hash, iv_hint, service, kind, created_at, revoked_at)`.

`pallet-reputation-tokens` (new): soulbound earned reputation tokens with minting policies indexed by behavior. Distinct from the governance-adjustable `reputation_bps` of the identity registry.

`pallet-substrate-attestation` (new): per Section 7, publishes LLM weights provenance on chain.

11.4. Credential vault flow

11.5. Crypto-shredding completeness

A key correctness property of the credential vault: revocation through master-key deletion must render all encrypted credentials inaccessible, *even if the ciphertext remains pinned to IPFS*.

Theorem 11.1 (Crypto-shredding completeness)**.** *Let k be a master key over a strongly-secure AEAD scheme (AES-256-GCM under the standard model). Let $\{C_1, \ldots, C_n\}$ be a set of ciphertexts $C_i = \mathsf{Enc}_k(m_i, iv_i)$ for messages m_i and unique nonces iv_i. Suppose k is securely erased from all storage*

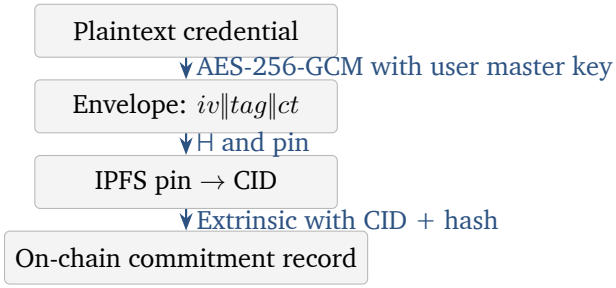

Figure 7: Credential vault encryption flow. Plaintext never leaves the client (*v0.2+*) or server boundary (*v0.1*); IPFS holds ciphertext only; chain holds commitment.

controlled by the holder. Then, for any computationally-bounded adversary $\mathcal{A}$ with access to all C_i but not to k,

$$\Pr[\mathcal{A}(C_1, \ldots, C_n) = m_i] \leq \frac{1}{|\mathcal{M}|} + \mathrm{negl}(\lambda),$$

where $\mathcal{M}$ is the plaintext domain and λ is the AEAD security parameter.

Proof. By the IND-CPA security of AES-256-GCM under the standard model. $\mathrm{negl}(\lambda)$ is the adversary's IND-CPA advantage at security parameter $\lambda = 256$. Crucially, AES-256-GCM's security guarantee binds the security to key secrecy, not to ciphertext exclusivity: public ciphertext does not weaken the bound. Erasing k thus removes all information-theoretic distinguishability from the chosen-message oracle. ■

Corollary 11.2 (CCPA crypto-shredding). *Compliance with right-to-erasure obligations (CCPA §1798.105, GDPR Art. 17) can be discharged by deleting the user's master key from all holder-controlled stores, without requiring unpinning of IPFS-resident ciphertexts.*

11.6. Sync mechanism

The Postgres+Redis mirror in the gov-app subscribes to chain events via Substrate's WebSocket RPC. Events from `identity-registry`, `axiom-attestation`, `pallet-reputation-tokens`, and `pallet-credential-vault` are decoded and applied to the relational schema; Redis caches are invalidated on the keyed indexes.

11.7. Verifier at action time

Theorem 11.3 (Verification-time accountability). *Let $\mathcal{I}$ be an identity attempting an action α at time t. The `verify_chain`$(\mathcal{I}, \alpha, t)$ helper returns $\top$ iff (i) $\mathcal{I}$ has an active L_3 key at t, (ii) $\mathcal{I}$'s $\prec$-ancestry terminates in finite steps at an S1 with a non-empty L_4 claim or in a stake-bonded entity, and (iii) $\mathcal{I}$'s L_5 reputation satisfies the action's minimum threshold θ_α.*

Proof. Conditions (i) and (iii) are checked by direct query against `identity-registry` and `pallet-reputation-tokens`. Condition (ii) terminates in finite steps by Theorem 6.2. ■

11.8. Costs and limitations

The architecture is engineering, and engineering has costs. We name them explicitly rather than letting them be discovered at integration time.

Signature storage: concrete numbers. Post-quantum signatures are substantially larger than classical ones. The relevant per-signature byte counts at NIST Level 5 / 128-bit post-quantum security:

Scheme	Sig (bytes)	Pub key (bytes)	Lane
Ed25519	64	32	classical
ECDSA-P256	64	64	classical
RSA-2048-PSS	256	270	classical
ML-DSA-87 (FIPS 204)	4,627	2,592	post-quantum
Falcon-512 (FN-DSA)	666	897	post-quantum
SPHINCS+-256f-simple (FIPS 205)	49,856	64	post-quantum (hash-only)

For Paraxiom's chosen surface (Falcon-512 for most extrinsics, SPHINCS+ for high-assurance attestations such as substrate provenance and validator-set commits), the dominant cost is Falcon's $\sim 10\times$ overhead per signature relative to Ed25519, plus periodic SPHINCS+-bearing extrinsics that consume ~ 50 KB each.

Chain-storage growth rate (worked estimate). Take a network sustaining 100 transactions per second, each carrying a single Falcon-512 signature. The signature surface alone contributes $100 \cdot 666\,\text{B} = 66.6\,\text{kB/s} \approx 5.8\,\text{GB/day} \approx 2.1\,\text{TB/year}$ *of chain state per validator*, before any non-signature payload. For the same network on Ed25519, the corresponding figure is $100 \cdot 64\,\text{B} = 6.4\,\text{kB/s} \approx 0.55\,\text{GB/day} \approx 200\,\text{GB/year}$. The $\sim 10\times$ overhead is real and must be budgeted across validator-side storage, archival nodes, and state-snapshot distribution.

A SPHINCS+-dominated deployment (validator-set commits every block at 500 ms block time, each carrying one SPHINCS+ signature) consumes an additional $2 \cdot 49\,856\,\text{B/s} \approx 100\,\text{kB/s} \approx 8.5\,\text{GB/day} \approx 3.1\,\text{TB/year}$. SPHINCS+ should therefore be applied sparingly, on the highest-assurance attestations only.

Hybrid signature complexity. Until the post-quantum ecosystem is mature, many deployments will run hybrid (classical + post-quantum) signatures. This doubles the per-extrinsic signing and verification work and the storage footprint — adding roughly $+64\,\text{B}$ (Ed25519 component) on top of every Falcon-bearing extrinsic. Net per-extrinsic chain bytes for a hybrid Falcon-512 + Ed25519 transaction: $666 + 64 = 730\,\text{B}$, a $1.1\times$ overhead on top of the PQ-only baseline. Cheap, and worth it during the transition window.

Realistic mitigations. Three techniques actually planned for Paraxiom-substrate deployments: (i) *selective PQ*: SPHINCS+ only for high-assurance strata (S5 cross-tenant founders, validator-set commits, substrate-attestation pallet), Falcon-512 for everything else, hybrid Ed25519 + Falcon for transitional interop with classical-only ecosystems; (ii) *state pruning*: validator nodes keep recent state in hot storage and archive older history to cold storage / IPFS-pinned snapshots, restoring on demand; (iii) *aggregated signatures* where the scheme permits: Falcon does not aggregate, but per-block SPHINCS+ commits can use Merkle aggregation of validator signatures to amortize the 50 KB cost across the validator set rather than per validator.

IPFS pinning: concrete numbers. The credential-vault flow places ciphertexts on IPFS, addressed by CID. A typical envelope-encrypted credential is 2–4 KB (AES-256-GCM header + ciphertext of a JSON-encoded credential payload). At commercial pinning rates as of May 2026 (Pinata: $\sim\$0.15$/GB-month; Filecoin storage providers via Web3.Storage: $\sim\$0.02$/GB-month for cold storage), an organization holding 10^4 active vault credentials sees a steady-state IPFS payload of ~ 30 MB, costing $\sim\$0.005$/month on Pinata or effectively free on Filecoin-tier infrastructure. The cost is dominated not by storage but by the operational burden of *guaranteeing pinning*: outage and replacement strategy, multi-provider redundancy, and recovery after a pinning-service failure are the real expense. We do not commit the architecture to a specific pinning strategy; we do commit to the property that an unpinned ciphertext is functionally crypto-shredded (Theorem 11.1), so operators retain the option of deliberate unpinning as a legitimate revocation mechanism.

Cross-substrate implementation complexity. Unifying human, machine, container, process, LLM substrate, LLM instance, and physical-good founding events under a single schema (Table 2) is elegant in the model and complex in realization: each substrate has its own attestation pipeline, hardware-root dependence, and failure modes. The unified type `Identity` carries optional fields per stratum; pallet-level validation must enforce stratum-specific invariants the type alone does not encode. This is recognized engineering debt of the architecture, not solved by the model.

Hypergraph state scaling. The identity-state hash $h_{\mathcal{J}}(t)$ (Theorem 2.14) re-hashes the serialization of all hyperedges incident on $\mathcal{J}$. For identities with many incident hyperedges (e.g., long-lived organizations or protocol bodies), the recomputation cost scales linearly in the cardinality. A merkleized implementation reduces this to logarithmic for incremental updates; we treat the construction as an implementation detail of `pallet-hypergraph-state` rather than a model-layer commitment.

Off-chain mirror invalidation. The Postgres+Redis mirror (Table 7) is rebuildable from chain plus IPFS but not always cheaply: a full rebuild walks the entire chain history and re-fetches all pinned payloads. Operators should plan for incremental event subscription, snapshotting, and graceful catch-up after extended downtime.

Operational costs. Beyond chain-bytes and storage, four recurring operational burdens deserve naming: (i) *key rotation*, which under PQ schemes requires regenerating the larger keypair (Falcon-512: 897 B public key; SPHINCS+: 64 B) and co-signing a rotation extrinsic — the standard frequency is annual for routine operations and immediate on suspected compromise, but the cost-per-rotation is dominated by the off-chain re-binding of the new key to relying parties; (ii) *recovery after crypto-shredding*: deliberate-erasure is a one-way operation by Theorem 11.1, so recovery requires re-onboarding through the original issuer (KYC issuer, manufacturer, etc.) rather than chain primitives — this is a feature, not a bug, but it imposes per-loss operational cost on the holder; (iii) *audit and compliance overhead*: the architecture produces a chain-of-custody artifact suitable for regulatory audit, but compiling auditor-ready reports (mapping chain events to regulatory categories, proving non-disclosure) is an off-chain ongoing task; (iv) *developer onboarding curve*: a competent Substrate developer needs $\sim$2 weeks to internalize the seven-stratum model, the privacy-tier mapping, and the credential-vault flow; consumer integrators (relying parties) need $\sim$1 week with the published SDKs to reach production-grade integration.

Governance of governance. Many architectural primitives (stratum lattice, governance-body taxonomy, privacy-tier mapping, ZK-augmented extrinsic set) require ongoing curation: how new strata are admitted, how governance bodies dispute or merge, how privacy tiers extend. The treatise specifies an initial set; the chain's upgrade mechanism (runtime spec version) is the operational tool for extending it. There is no escape from the recursive question "*who governs the chain that governs*."

What this architecture is deliberately *not* trying to be. Naming the negative space is as load-bearing as the positive claims:

- **Not a consumer wallet.** The treatise specifies a substrate; consumer-facing wallet UX, recovery flows, and KYC-onboarding integrations are deliberately out of scope.
- **Not a drop-in eIDAS or DIACC replacement.** The architecture interoperates with these frameworks at the wire-protocol layer (OID4VP, SD-JWT VC, ISO mdoc) but does not displace national trust frameworks; jurisdictional sovereignty is preserved by the L_4 anchor pattern (Theorem 2.3).
- **Not a recourse mechanism for non-consensual records.** Section 2.6 is restated here for clarity: this architecture provides no chain-native recourse against data-broker dossiers, surveillance records, algorithmic scoring, or employer dossiers. The applicable regulatory regimes (GDPR Art. 17, CCPA §1798.105, FCRA, EU AI Act) operate against such records; closing the asymmetry is research, not engineering.
- **Not a payment or DeFi layer.** The chain anchors identity; transferable assets and protocol-financial primitives belong to other layers and are not part of this specification.
- **Not a guarantee of correctness for downstream classifiers.** The substrate-attestation pallet (Section 7) binds outputs to weights and to a publisher; it does not attest that the weights themselves are correct, aligned, or unbiased. That is the publisher's responsibility, audited via separate mechanisms.

12. Privacy, anonymity, and user-controlled disclosure

The architecture described in Section 11, taken alone, is *transparency-by-default*: identity registrations, governance memberships, reputation tokens, and attestations are all visible on chain. A passive observer can reconstruct a substantial behavioral graph: who participates where, who has what reputation, who attested to whom.

This is the correct *baseline*: cryptographic accountability is worthless if it cannot be audited. But it is not the correct *end state*. A complete architecture must let an identity *choose what is revealed, to whom, and when*, while preserving the verifiability that makes the chain useful. This section formalizes that choice as the *privacy tier model* and proves the two key properties (selective-disclosure soundness, unlinkability under pairwise pseudonyms) that make user-controlled disclosure implementable.

12.1. The transparency baseline — honest acknowledgment

In the v0.1 and v0.2 designs of Section 11, the chain stores:

- Each identity's public key, owner `AccountId`, role, stratum, custody mode (*Identity*).
- For S1: a hash of an off-chain KYC bundle (*Civic claim*).
- For S2: the parent identity, tenant identifier, role (*Scope*).
- For S4: the parent, agent kind, `instance_anchor` (*Agency*).
- For S5: the multi-signature roots (*Organizational*).
- Per-body, per-token-kind reputation values (*Behavioral*).
- Per-action attestations with IPFS CIDs (*Provenance*).

A polynomial-time chain observer with no privileged access can correlate identifiers, build a per-identity participation graph across G-levels (Section 8), trace reputation accumulation, and link attestations to attesters. This is appropriate for many use cases (validator operations, public governance participation) and inappropriate for many others (employer-screening contexts, dissident speech, healthcare credentials, customer behavior on a multi-tenant SaaS).

The remedy is not to hide the chain; it is to let the identity-owner choose what is revealed in any given action and to provide cryptographic constructions that preserve verifiability under that choice.

12.2. The user-controlled disclosure principle

Definition 12.1 (User-controlled disclosure)**.** *User-controlled disclosure* means that for every action α attributable to identity $\mathcal{I}$, the holder of $\mathcal{I}$ chooses one of $\mathcal{I}$'s available *privacy postures* $\Pi \in \{\Pi_0, \Pi_1, \Pi_2, \Pi_3, \Pi_4\}$ at the moment of action. The chain enforces:

1. the action's verifier requirement (what must be proven for the action to succeed),
2. the holder's chosen posture, provided the posture's primitives can satisfy the requirement.

The chain does not enforce a global posture per identity. Different actions by the same identity may use different postures.

12.3. The privacy tier model

Definition 12.2 (Privacy tier)**.** The *privacy tier* of a claim is an element of $\mathbb{P} = \{\Pi_0, \Pi_1, \Pi_2, \Pi_3, \Pi_4\}$ defined as follows.

Π_0 ***Public*** All claim attributes and the holder's canonical identifier are revealed on chain.

Π_1 ***Pseudonymous*** The claim is signed by a stealth / derived account unlinked to the holder's canonical L_4 identity; attribute values are revealed in cleartext.

Π_2 ***Selective disclosure*** The claim is presented as a credential (SD-JWT VC or ISO mdoc selective disclosure, or BBS+ signature) in which only the attributes the holder elects to reveal are decrypted; the unrevealed attributes remain cryptographically committed.

Π_3 ***Anonymous (set membership)*** The claim is proven as "some member of authorized set S signed this" via ring signature, group signature, or ZK proof of set membership; the individual signer's identity is hidden among $|S|$ candidates.

Π_4 ***Zero-knowledge*** The claim is presented as a ZK proof of a property ϕ over $\mathcal{I}$'s secret state; neither the secret state nor the holder's identifier is revealed. Examples: $\text{rep}(\mathcal{I}, B) \geq \theta$ as a ZK range proof; $\mathcal{I} \in B$ as a ZK proof of membership; "$\mathcal{I}$ holds a credential issued by X" as a ZK proof of credential possession.

12.4. Primitives by tier

Π_1 **pseudonymous.** A stealth account is a fresh keypair derived from the holder's master secret via a context-specific KDF: $\text{sk}_{\text{ctx}} = \text{KDF}(\text{sk}_{\text{master}}, \text{ctx})$. The chain sees sk_{ctx}'s public key but not the link to $\text{sk}_{\text{master}}$.

Π_2 **selective disclosure.** The credentials in `pallet-credential-vault` can be issued in SD-JWT VC or BBS+ form. At presentation time, the holder produces a *derived* credential exposing only the attributes the verifier needs. The on-chain commitment remains the same; the `attest_use` extrinsic records that a presentation occurred but not which attributes were revealed.

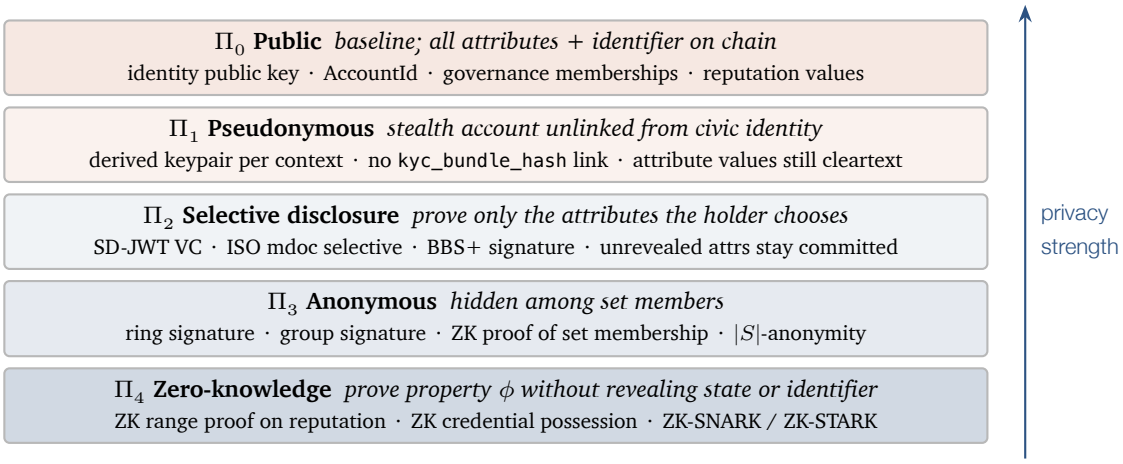

Figure 8: Privacy tier model. The holder of an identity chooses a posture $\Pi \in \{\Pi_0, \ldots, \Pi_4\}$ per action. The chain enforces that the holder's chosen primitives suffice to satisfy the verifier's requirement; nothing forces a holder to a uniform posture across actions.

Π_3 **anonymous (set membership).** The most useful instance is *anonymous voting*: in a governance body B with membership $\mathcal{M}_B$, a voter proves "I am some member of $\mathcal{M}_B$" via a ring signature over the set's public keys, or via a ZK-SNARK proof of inclusion in $\mathcal{M}_B$ (the latter scales better for $|\mathcal{M}_B| > 100$). The vote-counting runtime verifies the membership proof and records the vote against the body, not against the voter.

Π_4 **zero-knowledge.** Used where the property to be proven is parameterized rather than a mere set membership. Examples:

- **Reputation thresholds:** $\text{rep}(\mathcal{I}, B) \geq \theta$ as a ZK range proof. The verifier learns only that the threshold is met. Implemented over Bulletproofs or via Stark-friendly hash commitments depending on proof size constraints.
- **Credential possession:** "$\mathcal{I}$ holds a credential issued by X" as a ZK proof of signature-knowledge. BBS+ signatures admit efficient selective-disclosure proofs natively.
- **Age / threshold attributes:** $\text{DOB}(\mathcal{I}) \leq t - 18\,\text{years}$ as a ZK range proof. Standard in mobile-driver-license (mDL) deployments via ISO/IEC 18013-5 selective disclosure profile.
- **Anonymous attestation:** "some valid agent of class X produced this output" via Direct Anonymous Attestation (DAA)-style construction. Used for agent swarms (Section 6) where individual instance identity is not relevant.

Paraxiom's stack already has the primitive building blocks: SPHINCS+/Falcon (PQ signature surface), `pedersen-commitment` pallet (for commitments), STARK proofs deployed in Drista [8]. The remaining work is to compose these into ZK-augmented variants of the identity-bearing extrinsics in Section 11.

12.5. Selective disclosure soundness

Theorem 12.3 (Selective disclosure soundness)**.** *Let σ be a BBS+ or SD-JWT VC presentation from holder $\mathcal{I}$ disclosing attributes $A_{\text{disc}} \subseteq A_{\text{full}}$ of a credential issued by issuer X with public key pk_X. If a verifier V accepts σ, then with probability at least $1 - \text{negl}(\lambda)$:*

1. *Issuer X has actually signed a credential with the same values for A_{disc};*
2. *The values for the undisclosed attributes $A_{\text{full}} \setminus A_{\text{disc}}$ are unknown to V beyond the prior distribution over those attributes.*

Proof sketch. Soundness of (1) follows from the SUF-CMA security of the underlying BBS+ (or, in the SD-JWT case, the JWS signature scheme) under the chosen pairing assumption (typically SXDH). Zero-knowledge of (2) follows from the salted-hash construction (SD-JWT) or the randomization of the BBS+ signature, both of which yield indistinguishability of unrevealed attribute values from independent uniform draws conditional on the full credential being well-formed [11, 13]. ■

12.6. Unlinkability under pairwise pseudonyms

Theorem 12.4 (Unlinkability under pairwise pseudonyms)**.** *Let $\text{sk}_{\text{master}}$ be a holder's master secret, let ctx_a and ctx_b be two distinct context identifiers, and let $\text{sk}_a = \text{KDF}(\text{sk}_{\text{master}}, \text{ctx}_a)$, $\text{sk}_b = \text{KDF}(\text{sk}_{\text{master}}, \text{ctx}_b)$ be the derived context-specific keys. Let pk_a, pk_b be the corresponding public keys. For any polynomial-time adversary $\mathcal{A}$ given pk_a, pk_b but not $\text{sk}_{\text{master}}$:*

$$\Pr[\mathcal{A}(\text{pk}_a, \text{pk}_b) = 1 \mid \textit{same holder}] - \Pr[\mathcal{A}(\text{pk}_a, \text{pk}_b') = 1 \mid \textit{different holders}] \leq \text{negl}(\lambda),$$

where pk_b' is a freshly-drawn public key from an independent holder. That is, $\mathcal{A}$ cannot distinguish whether two pseudonyms share a master with non-negligible advantage.

Proof sketch. Reduces to the pseudo-randomness of the KDF. If $\mathcal{A}$ distinguishes the cases with advantage ϵ, we construct a distinguisher against the KDF output distribution: query the KDF oracle on ctx_a and ctx_b for either a shared master (real case) or independent draws (random case); the public keys are deterministic functions of the secrets; pass them to $\mathcal{A}$ and output $\mathcal{A}$'s output. The KDF's pseudo-random output distribution under standard assumptions [1] bounds $\epsilon \leq \text{negl}(\lambda)$. ■

12.7. The mixed-tier leak — privacy is not a default state

Remark 12.5 (Mixed-tier leak)**.** Privacy guarantees do not compose trivially across tiers used by the same holder. Suppose holder $\mathcal{I}$ takes action α_1 at posture Π_0 (publicly revealing identity) and action α_2 at posture Π_4 (zero-knowledge). If α_1 and α_2 share side-channel observables (timing, network origin, social-graph neighbors, distinctive content), an adversary may correlate them despite the cryptographic anonymity of α_2. Maintaining the benefit of Π_3 or Π_4 thus requires:

1. Consistent posture across linkable contexts.
2. Operational discipline at lower layers (mixnets for network origin, time-uniform action submission, persona separation at the application layer).

This is a structural property, not a flaw in the cryptography. Drista's PQ mesh and the planned `qrsync` sync protocol address the network-layer side; persona separation is the user- facing side.

12.8. Compliance-aware privacy (zk-KYC)

A frequent regulatory bind: a relying party must establish that a counterparty has been KYC-verified by an authorized issuer, but the counterparty does not want to reveal their full identity to the relying party. The privacy tier model resolves this via a Π_4 construction we call *zk-KYC*:

1. Issuer X (a KYC provider) issues a credential to holder $\mathcal{I}$ with attributes including `kyc_verified=true`, `jurisdiction=CA-QC`, `verification_date=...`.
2. The credential is committed on chain via `pallet-credential-vault` per Section 11.
3. At presentation time to relying party R, $\mathcal{I}$ produces a ZK proof of the predicate "$\exists\, c : c \in \text{issuer}(X) \wedge c.\texttt{kyc\_verified} = \text{true} \wedge c.\texttt{jurisdiction} = \text{CA-QC} \wedge c.\texttt{holder} = \mathcal{I}$" without revealing c's other attributes.
4. Relying party R verifies the proof against X's public key and $\mathcal{I}$'s on-chain commitment.

R learns: $\mathcal{I}$ has been KYC-verified by an authorized issuer in Quebec. R does NOT learn: $\mathcal{I}$'s name, date of birth, address, document numbers, or any other attribute of the credential. This satisfies the regulatory requirement without the privacy cost.

12.9. ZK-augmented pallet roadmap

The pallets of Section 11 each admit a ZK-augmented variant; the runtime can accept both classical and ZK-augmented extrinsics, letting the user choose per action.

- `identity-registry::register_identity_zk`: register an identity using a pseudonymous account; the canonical `kyc_bundle_hash` field is replaced by a Pedersen commitment + ZK proof of inclusion in an issuer's attested set.
- `pallet-credential-vault::present_zk`: replaces `attest_use` with a presentation extrinsic that takes a ZK proof of credential possession + selective disclosure of named attributes.
- `pallet-governance-bodies::vote_anonymous`: vote on a body's decision by submitting a ring proof or ZK-SNARK proof of membership in $\mathcal{M}_B$; the vote is recorded against B without revealing which member.
- `pallet-reputation-tokens::prove_threshold`: produce a ZK range proof that $\text{rep}(\mathcal{I}, B) \geq \theta$ for verifier consumption; the chain confirms the proof but does not reveal the exact value.

- `axiom-attestation::attest_anonymous`: emit an attestation under a Π_3 ring signature over an issuer set, appropriate for agent-swarm attestations (Section 6 §6.3).

These ZK-augmented variants ship in v0.3 of the architecture (post-v0.2 self-sovereign custody). The v0.1 / v0.2 chain accepts only Π_0 and Π_1 extrinsics; the v0.3 chain extends the extrinsic set without breaking v0.1 / v0.2 records. Forward compatibility is preserved.

13. Future scenarios

We project three scenarios for the trajectory of identity past 2030.

13.1. Dissolution

Identity collapses if (a) AI agents proliferate faster than accountability chains can be maintained; (b) CRQC arrives before PQC migration completes for civic-identity infrastructure; (c) states lose capacity to issue civic identity; (d) privacy norms preclude reputation tracking. Non-zero probability; the canonical reason this work is timely.

13.2. Fragmentation

Identity persists but jurisdictionalizes: each major bloc operates its own infrastructure with limited interoperability. This is the trajectory the world is presently on. Paraxiom's chain-native, state-agnostic substrate functions as a bridge across fragmented zones, attesting to jurisdictional claims when needed via `kyc_bundle_hash` but not depending on any single jurisdiction's continued existence.

13.3. Augmentation

Identity extends: humans, agents, swarms, organizations, protocols all carry first-class cryptographic identity with explicit accountability chains and portable reputation. The five-stratum design of Theorem 6.1 is a first draft of an augmented identity ontology; further strata (swarm, protocol, hybrid) extend the schema.

The outcome among these three depends not on inevitability but on what is built. Dissolution occurs by default if no one builds the infrastructure; fragmentation if states build it without interoperability; augmentation if deliberate choice produces post-quantum, decentralized, accountability-preserving substrates.

14. Conclusion

Identity is engineering. The accountability chain does not extend itself; the post-quantum signatures do not sign themselves; the strata do not enumerate themselves. The substrates we now share with machines and AI agents will, by the early 2030s, demand identity infrastructure that survives a quantum-cryptographic transition, that scales without requiring social acquaintance, that holds open the possibility of new kinds of accountability loci we cannot fully enumerate today.

We have stated identity formally as a layered accountability chain above a founding individuation event, proven seven invariants (founding uniqueness, tier monotonicity, stratum well-foundedness, LLM instance non-identity, cryptographic survival, crypto-shredding completeness, and verification-time accountability), surveyed the 2026 deployed landscape and located Paraxiom's three uncontested lanes (PQ civic-grade, agent cryptographic identity, model substrate attestation), and specified the on-chain plus IPFS plus mirror architecture that realizes the model.

The question with which this treatise was commissioned — "will we have identity in the future?" — admits a precise answer. Yes, if built. The pallets specified in Section 11 are the construction.

A. Detailed proofs

A.1. Founding uniqueness (full)

Full proof of Theorem 2.8. We argue from Theorem 2.3, Theorem 2.1, and Theorem 2.5.

Let $\mathcal{I} = (\mathcal{S}, \mathcal{F}, \mathcal{A}, \mathcal{R})$ be an identity locus and suppose for contradiction that there exist two founding-event candidates $\mathcal{F}_1 = (s, t_0^{(1)}, \pi_1)$ and $\mathcal{F}_2 = (s, t_0^{(2)}, \pi_2)$, both consistent with $\mathcal{I}$ in the sense that every claim in $\mathcal{A}$ post-dates the candidate's timestamp.

Case 1: $t_0^{(1)} < t_0^{(2)}$. Then there exist points in time $t \in (t_0^{(1)}, t_0^{(2)})$ at which $\mathcal{S}$ was already individuated relative to $\mathcal{F}_1$ but, by hypothesis, not yet individuated relative to $\mathcal{F}_2$. Any claim made in this interval — and Theorem 2.5 permits such claims to exist in $\mathcal{A}$ relative to $\mathcal{F}_1$ — would violate Theorem 2.5 relative to $\mathcal{F}_2$. Either $\mathcal{F}_2$ does not consistently anchor $\mathcal{I}$, or no claims exist in the interval (in which case $t_0^{(2)}$ can be taken as $t_0^{(1)}$ without loss of information). In both sub-cases we collapse to a single founding event.

Case 2: $t_0^{(1)} = t_0^{(2)}$. Then both events name the same moment of individuation of the same substrate; up to the witness set, they coincide. The witness sets may be merged: $\pi = \pi_1 \cup \pi_2$ produces the most informative single $\mathcal{F}$, and the two-event presentation reduces to a single-event presentation.

In both cases the two candidates collapse to one, contradicting the assumption of distinctness. ■

A.2. Stratum well-foundedness (full)

Full proof of Theorem 6.2. We rely on three facts about the chain registration protocol:

1. Every S2 or S4 registration extrinsic requires a non-null `parent_identity` field; the chain validates that the named parent exists prior to applying the registration.
2. S5 registration requires either a multi-signature attestation of S1 founders or a jurisdictional L_4 claim. Both terminate finitely at S1 or at a jurisdictional authority treated as an S1 surrogate.
3. Every identity in `identity-registry` carries a `created_at_block` field, strictly less than the block in which any of its children are registered.

Suppose for contradiction an infinite descending chain $\mathcal{I}_0 \succ \mathcal{I}_1 \succ \mathcal{I}_2 \succ \ldots$. By fact 3, block$(\mathcal{I}_0) >$ block$(\mathcal{I}_1) >$ block$(\mathcal{I}_2) > \ldots$, an infinite strictly decreasing sequence in $\mathbb{N}$, which is impossible.

Thus every descending chain terminates in finite steps. By facts 1 and 2, the terminus is either an S1 or an S1 surrogate. In either case, finite-step termination at an S1-equivalent is guaranteed. ■

A.3. Cryptographic survival (full)

Full proof of Theorem 9.1. **Forward direction.** For $\Sigma \in \Sigma_{\text{PQ}}$ each scheme has been demonstrated EUF-CMA-secure against any polynomial-time quantum adversary under the standard model, per the security reductions underlying NIST FIPS 203–205. Let $\mathcal{Q}$ be such a quantum adversary with running time bounded by poly(λ). The EUF-CMA security definition states:

$$\Pr[\text{EUF-CMA}_{\mathcal{Q}}(\Sigma, \lambda) = 1] \leq \text{negl}(\lambda).$$

This implies that the probability of any forgery against any single claim a_i in $\mathcal{A}$ is at most negl(λ). By a union bound over the n claims, the probability of any forgery anywhere in $\mathcal{A}$ is at most $n \cdot \text{negl}(\lambda)$, which remains negligible for any polynomial-size $\mathcal{A}$.

Reverse direction. For $\Sigma' \in \Sigma_{\text{cl}}$, Shor's algorithm [4] solves the discrete logarithm problem in $\mathbb{Z}_p^*$ and on elliptic curves over $\mathbb{F}_p$ in time polynomial in $\log p$. Given a CRQC of size sufficient to implement Shor at the relevant key size, the adversary can recover sk from pk with probability $1 - \text{negl}'(\lambda')$, after which forgery is trivial via Sign_{sk}. The probability bound on forgery is hence at least $1 - \text{negl}'(\lambda')$, where negl$'(\lambda')$ is characterized by the CRQC's residual error, dominated by the classical key length λ'.

■

A.4. Crypto-shredding completeness (full)

Full proof of Theorem 11.1. We reduce to the IND-CPA security of AES-256-GCM. Suppose for contradiction that $\mathcal{A}$ achieves

$$\Pr[\mathcal{A}(C_1, \ldots, C_n) = m_i] > \frac{1}{|\mathcal{M}|} + \epsilon$$

for non-negligible ϵ, for some target index i. Construct an IND-CPA adversary $\mathcal{B}$ against AES-256-GCM as follows: $\mathcal{B}$ receives an IND-CPA challenge oracle, queries it on all $\{m_1, \ldots, m_n\}$ to obtain ciphertexts identically distributed to those $\mathcal{A}$ would receive, then runs $\mathcal{A}$ on these ciphertexts. The output of $\mathcal{B}$ is $\mathcal{A}$'s output.

If $\mathcal{A}$ recovers m_i with advantage ϵ over the $1/|\mathcal{M}|$ baseline, then $\mathcal{B}$ distinguishes the IND-CPA challenge with the same advantage. By IND-CPA security of AES-256-GCM, this advantage is bounded by $\mathrm{negl}(\lambda)$ where $\lambda = 256$ is the AES key length. Hence $\epsilon \leq \mathrm{negl}(\lambda)$, contradicting the supposed non-negligibility. ■

B. Pallet API specifications

B.1. `identity-registry` (extended)

```
pub enum Stratum { S1Person, S2Scoped, S4AgentInstance, S5CrossTenant }

pub enum CustodyMode { SelfSovereign, NcCustodial, TenantCustodial }

pub enum IdentityRole {
    Shield, Agent, OracleReporter, Validator,
    Citizen, Decideur, Fournisseur, Service,
}

pub enum IdentityStatus {
    Active, Suspended, Revoked, Deceased, Decommissioned, Dissolved
}

/// Register an S1 identity, optionally with KYC bundle.
pub fn register_identity_s1(
    origin: OriginFor<T>,
    public_key: BoundedVec<u8, ConstU32<2048>>,
    kyc_bundle_hash: Option<[u8; 32]>,
    custody: CustodyMode,
) -> DispatchResult;

/// Delegate to a scoped S2 identity from a caller S1.
pub fn delegate_to_scope(
    origin: OriginFor<T>,
    tenant_id: BoundedVec<u8, ConstU32<64>>,
    role: IdentityRole,
) -> DispatchResult;

/// Register an S4 agent instance under a parent identity.
```

```
pub fn register_agent_instance(
    origin: OriginFor<T>,
    parent_identity: u64,
    agent_kind: BoundedVec<u8, ConstU32<64>>,
    public_key: BoundedVec<u8, ConstU32<2048>>,
    instance_anchor: [u8; 32],
) -> DispatchResult;

/// Verify the accountability chain for an action.
pub fn verify_chain(
    identity_id: u64,
    action_kind: ActionKind,
    block: BlockNumberFor<T>,
) -> Result<ChainVerdict, DispatchError>;
```

B.2. `pallet-governance-bodies`

```
pub enum GovernanceLevel {
    G0Self, G1Domestic, G2Project, G3Organization,
    G4Federation, G5Civic, G6Supranational, G7Protocol,
}

pub enum ScopeClaim {
    DecisionDomain(BoundedVec<u8, ConstU32<64>>),
    ResourceControl(BoundedVec<u8, ConstU32<64>>),
    AttestationAuthority,
    MembershipAdmin,
    Custom(BoundedVec<u8, ConstU32<128>>),
}

pub struct GovernanceBody<AccountId, BlockNumber> {
    pub id: u64,
    pub level: GovernanceLevel,
    pub name: BoundedVec<u8, ConstU32<128>>,
    pub parent_body: Option<u64>,
    pub charter_hash: [u8; 32],
    pub charter_ipfs_cid: BoundedVec<u8, ConstU32<128>>,
    pub chartered_scope: BoundedVec<ScopeClaim, ConstU32<16>>,
    pub parent_signature: BoundedVec<u8, ConstU32<2048>>,
    pub status: BodyStatus,
    pub created_at_block: BlockNumber,
    pub jurisdictional_anchor: Option<JurisdictionRef>,
}

pub fn register_body(
    origin: OriginFor<T>,
    level: GovernanceLevel,
    name: BoundedVec<u8, ConstU32<128>>,
    parent_body: Option<u64>,
    chartered_scope: BoundedVec<ScopeClaim, ConstU32<16>>,
    charter_ipfs_cid: BoundedVec<u8, ConstU32<128>>,
    parent_signature: BoundedVec<u8, ConstU32<2048>>,
```

```
) -> DispatchResult;
// Runtime enforces Theorem 8.3: chartered_scope SUBSET parent_chartered_scope.

pub fn add_member(origin: OriginFor<T>, body_id: u64, member: u64) -> DispatchResult;
pub fn remove_member(origin: OriginFor<T>, body_id: u64, member: u64) -> DispatchResult;
pub fn terminate_body(origin: OriginFor<T>, body_id: u64, successor: Option<u64>) ->
    DispatchResult;
```

B.3. `pallet-credential-vault`

```
pub struct Commitment<BlockNumber> {
    pub holder_identity: u64,
    pub service_name: BoundedVec<u8, ConstU32<64>>,
    pub credential_kind: CredentialKind,
    pub ciphertext_cid: BoundedVec<u8, ConstU32<128>>,
    pub ciphertext_hash: [u8; 32],
    pub iv_hint: [u8; 12],
    pub created_at: BlockNumber,
    pub revoked_at: Option<BlockNumber>,
}

pub fn commit(
    origin: OriginFor<T>,
    service_name: BoundedVec<u8, ConstU32<64>>,
    credential_kind: CredentialKind,
    ciphertext_cid: BoundedVec<u8, ConstU32<128>>,
    ciphertext_hash: [u8; 32],
    iv_hint: [u8; 12],
) -> DispatchResult;

pub fn revoke(
    origin: OriginFor<T>,
    commitment_id: u64,
) -> DispatchResult;

pub fn attest_use(
    origin: OriginFor<T>,
    commitment_id: u64,
    caller_context: BoundedVec<u8, ConstU32<128>>,
) -> DispatchResult;
```

B.4. `pallet-substrate-attestation`

```
pub struct SubstrateAttestation<BlockNumber> {
    pub weights_hash: H256,
    pub training_run_merkle: H256,
    pub arch_spec_hash: H256,
    pub publisher: u64,                // identity-registry ID
    pub publisher_sig: BoundedVec<u8, ConstU32<2048>>,
    pub ipfs_cid_card: BoundedVec<u8, ConstU32<128>>,
```

```
    pub published_at: BlockNumber,
    pub revoked_at: Option<BlockNumber>,
}

pub fn attest_substrate(
    origin: OriginFor<T>,
    weights_hash: H256,
    training_run_merkle: H256,
    arch_spec_hash: H256,
    publisher_falcon_sig: BoundedVec<u8, ConstU32<2048>>,
    ipfs_cid_card: BoundedVec<u8, ConstU32<128>>,
) -> DispatchResult;
```

C. ZK-augmented pallet extrinsics (v0.3)

The v0.3 chain extends the v0.2 pallet set with privacy-preserving extrinsics. Both classical and ZK-augmented variants coexist; the user chooses per action per Theorem 12.1.

C.1. ZK-augmented identity registration

```
pub struct PedersenCommitment {
    pub commitment: [u8; 32],
    pub blinding_proof_hint: [u8; 32],
}

pub struct ZkKycProof {
    /// Proves: commitment opens to (kyc_verified=true, jurisdiction, holder=self)
    /// for some credential issued by an authorized KYC issuer.
    pub proof_bytes: BoundedVec<u8, ConstU32<8192>>,
    pub authorized_issuer_set_root: H256,
}

pub fn register_identity_zk(
    origin: OriginFor<T>,
    public_key: BoundedVec<u8, ConstU32<2048>>,
    kyc_commitment: PedersenCommitment,
    zk_kyc_proof: ZkKycProof,
    custody: CustodyMode,
) -> DispatchResult;
```

C.2. ZK credential presentation

```
pub struct ZkPresentationProof {
    /// BBS+ / SD-JWT-style selective disclosure proof.
    pub proof_bytes: BoundedVec<u8, ConstU32<8192>>,
    /// The disclosed attributes (revealed in cleartext per holder's choice).
    pub disclosed_attrs: BoundedVec<(BoundedVec<u8, ConstU32<64>>, BoundedVec<u8, ConstU32
        <256>>), ConstU32<32>>,
```

```
}

pub fn present_zk(
    origin: OriginFor<T>,
    commitment_id: u64,
    relying_party_context: BoundedVec<u8, ConstU32<128>>,
    proof: ZkPresentationProof,
) -> DispatchResult;
```

C.3. Anonymous governance vote

```
pub struct AnonymousMembershipProof {
    /// Proves: signer in {membership set of body_id at proof_block}.
    pub proof_bytes: BoundedVec<u8, ConstU32<16384>>,
    pub proof_block: u64,
    pub nullifier: [u8; 32],  // prevents double-voting
}

pub fn vote_anonymous(
    origin: OriginFor<T>,
    body_id: u64,
    decision_id: u64,
    choice: VoteChoice,
    membership_proof: AnonymousMembershipProof,
) -> DispatchResult;
// Runtime stores nullifier to prevent the same member from voting twice
// without revealing which member.
```

C.4. ZK reputation threshold proof

```
pub struct ReputationRangeProof {
    /// Bulletproof or STARK proof that rep(holder, body) >= threshold.
    pub proof_bytes: BoundedVec<u8, ConstU32<4096>>,
    pub commitment_to_holder: PedersenCommitment,
}

pub fn prove_threshold(
    origin: OriginFor<T>,
    body_id: u64,
    token_kind: BoundedVec<u8, ConstU32<64>>,
    threshold: u32,
    proof: ReputationRangeProof,
) -> Result<bool, DispatchError>;
// Verifier-side helper; does not write to state.
```

D. Lean formalization excerpt

The following is the verbatim Lean 4 proof of Theorem 6.2 ("every non-S1 identity reaches an S1 ancestor in finite steps") from the companion repository [24], file `IdentityArchitecture/Stratum.lean`. It is the most substantive of the four fully-deductive proofs in the formalization: a strong-induction argument over chain block height showing that the parent-pointer relation terminates at S1 in finitely many steps. The proof depends only on Mathlib v4.27.0 primitives (`Nat.le_zero`, `Nat.le_refl`, `omega`) and the `ChainDiscipline` structure declared earlier in the same file; no cryptographic or probabilistic axioms appear.

```
theorem stratum_wellfoundedness
    (R : Registry) (hCD : ChainDiscipline R) :∀
     I ∈ R, ∃ n J, J ∈ R ∧ J.stratum = Stratum.s1Person ∧ AncestorAt R n I J := by
  -- Strengthen: prove the statement for all identities with blockHeight ≤ k,
  -- by induction on k.
  suffices h : ∀ k : Nat, ∀ I ∈ R, I.blockHeight ≤ k →∃
       n J, J ∈ R ∧ J.stratum = Stratum.s1Person ∧ AncestorAt R n I J by
    intro I hI
    exact h I.blockHeight I hI (Nat.le_refl _)
  intro k
  induction k with
  | zero =>
    intro I hI hbh
    by_cases hS1 : I.stratum = Stratum.s1Person
    · exact ⟨0, I, hI, hS1, AncestorAt.refl ⟩I
    · -- I has a parent P with P.blockHeight < I.blockHeight ≤ 0 — impossible
      obtain ⟨pid, hpid, P, hP_in, ⟩hP_id := hCD.nons1_has_parent I hI hS1
      have hP_lt : P.blockHeight < I.blockHeight :=
        hCD.parent_height_strict I hI pid hpid P hP_in hP_id
      have hI0 : I.blockHeight = 0 := Nat.le_zero.mp hbh
      omega
  | succ n ih =>
    intro I hI hbh
    by_cases hS1 : I.stratum = Stratum.s1Person
    · exact ⟨0, I, hI, hS1, AncestorAt.refl ⟩I
    · obtain ⟨pid, hpid, P, hP_in, ⟩hP_id := hCD.nons1_has_parent I hI hS1
      have hP_lt : P.blockHeight < I.blockHeight :=
        hCD.parent_height_strict I hI pid hpid P hP_in hP_id
      have hP_le_n : P.blockHeight ≤ n := by omega
      obtain ⟨m, J, hJ_in, hJ_S1, ⟩hAncP := ih P hP_in hP_le_n
      refine ⟨m + 1, J, hJ_in, hJ_S1, ?⟩_
      have hpidP : I.parentId = some P.id := hP_id ▸ hpid
      exact AncestorAt.step hI hP_in hpidP hAncP
```

Informal proof sketch. We strengthen the statement to: for every k, every identity I with block height $\leq k$ reaches an S1 ancestor in finite steps. We induct on k. *Base case* $k = 0$: if I is S1, the 0-step ancestor is I itself; if I is non-S1, the chain discipline forces I to have a parent P with P.blockHeight $<$ I.blockHeight ≤ 0, an impossibility in $\mathbb{N}$. *Inductive step* $k = n + 1$: if I is S1, done; otherwise I's parent P has P.blockHeight $\leq n$ and the induction hypothesis gives an S1 ancestor of P in finite steps, which is also an S1 ancestor of I at one greater depth. The Lean proof above is the exact mechanization of this argument.

The six other theorems in the formalization (Table 1) follow similar structures; they are viewable in full in the companion repository.

E. Sources

References

[1] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.* U.S. Department of Commerce, 13 August 2024.

[2] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard.* U.S. Department of Commerce, 13 August 2024.

[3] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard.* U.S. Department of Commerce, 13 August 2024.

[4] P. W. Shor. *Algorithms for quantum computation: discrete logarithms and factoring.* Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994.

[5] European Parliament and Council. *Regulation (EU) 2024/1183 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework (eIDAS 2.0)*. Official Journal of the European Union, May 2024.

[6] Anthropic. *Model Context Protocol Specification, Release 2025-11-25*. https://modelcontextprotocol.io/

[7] Google. *Agent-to-Agent (A2A) Protocol.* Published April 2025.

[8] Paraxiom Technologies Inc. *PARAXIOM-MODIFICATIONS.md: canonical fork-change manifest for polkadot-sdk and parity-scale-codec.* QuantumHarmony repository, 2026.

[9] Paraxiom Technologies Inc. *pallet-identity-registry source code.* QuantumHarmony pallets directory, 2026.

[10] American Association of Motor Vehicle Administrators. *Mobile Driver License Implementation Guidelines v1.5*. May 2026.

[11] World Wide Web Consortium. *Verifiable Credentials Data Model 2.0 (W3C Recommendation)*. 15 May 2025.

[12] World Wide Web Consortium. *Decentralized Identifiers (DIDs) v1.1 (W3C Candidate Recommendation Snapshot)*. 5 March 2026.

[13] OpenID Foundation. *OpenID for Verifiable Presentations 1.0 (Final)*. July 2025.

[14] OpenID Foundation. *OpenID for Verifiable Credential Issuance 1.0 (Final)*. July 2025.

[15] National Security Agency. *Commercial National Security Algorithm Suite 2.0.* U.S. Department of Defense, September 2022 (updated 2025).

[16] European Union Agency for Cybersecurity. *Post- Quantum Cryptography: Current State and Quantum Mitigation.* ENISA, 2025.

[17] Trusted Computing Group. *Device Identifier Composition Engine (DICE) Architectures.* TCG Specification, 2024 revision.

[18] Policy Circle. *Aadhaar authentication failures persist after a decade.* June 2025.

[19] Linux Foundation. *Agentic AI Foundation launch announcement.* December 2025.

[20] S. Cormier. *Augmented Democracy: A Machine-Based Societal Model for Curbing Citizen Cynicism.* August 2017. Source repository: `https://github.com/sylvaincormier/augmented-democracy` (initial commit 2018-04-15, material drafted August 2017).

[21] E. G. Weyl, P. Ohlhaver, and V. Buterin. *Decentralized Society: Finding Web3's Soul.* SSRN, May 2022.

[22] Chainlink Labs. *Dynamic NFTs: On-chain assets that evolve with oracle inputs.* Technical brief, 2023–2024.

[23] J. Hwang, B. Burns, S. Vroegop, V. Mahabadi, and contributors. *ERC-6551: Non-fungible Token Bound Accounts.* Ethereum Improvement Proposal, finalized 2024.

[24] S. Cormier. *identity-architecture-lean: Lean 4 formalization of the seven identity-architecture theorems.* Lean 4.27.0 + Mathlib v4.27.0, May 2026. Source repository: `https://github.com/Paraxiom/identity-architecture-lean`.

Document version: 1.0, May 22, 2026. *Companion materials*: `IDENTITY-ARCHITECTURE.md` (narrative version); `IDENTITY-PALLETS-DESIGN.md` (engineering specification); `IDENTITY-SUBSTRATES.md` (substrate spec + ephemeral identities); `GOVERNANCE-ROADMAP.md` §10 (product roadmap). *Repository*: `https://github.com/Paraxiom/identity-architecture`.

Treatise primitive	W3C 2025–2026 status	Where defined here
Post-quantum signature surface (SPHINCS+, Falcon-512)	Mentioned as future work in VC 2.0; no normative requirement	Section 9, Theorem 9.1
Substrate attestation (LLM weights, hardware)	Not addressed	Section 7, `pallet_substrate_attestation`
Six-layer accretion model (L_0–L_5)	Not formalized; VC has credentials, not layers	Section 3
Permanence-tier hierarchy (Tier 0–3)	Not formalized	Section 5, Theorem 5.2
Five identity strata (S1–S5) with parent-identity well-foundedness	Not formalized; VC has subject/issuer/holder, not strata	Theorem 6.1, Theorem 6.2
LLM instance non-identity	Not addressed	Theorem 7.3
Self-assertion as formal layer (signed by $sk(\mathcal{I})$)	Implicit in VC "holder" role; not distinguished from external attestation	Theorem 2.10
Non-consensual third-party identity (out of scope, named)	Not addressed	Section 2.6
Hypergraph identity state ($\mathcal{H}$, $h_{\mathcal{I}(t)}$)	RDF graph is binary; hyperedges via reification only	Theorem 2.12, Theorem 2.14
Physical-good identity as a first-class founding domain	Not addressed; NFT lane is separate ecosystem (Aura, Arianee)	Section 4.5
Crypto-shredding completeness (right-to-erasure via key destruction)	Not addressed; status-list revocation is the W3C primitive	Theorem 11.1
Verification-time accountability (finite-step chain verification)	Not formalized	Theorem 11.3
Governance bodies as on-chain objects with chartered scope bounding	Not addressed	Theorem 8.2, Theorem 8.4
Multi-level participation independence & reputation non-transitivity	Not addressed	Theorem 8.5, Theorem 8.6
Five-tier privacy model (Π_0–Π_4) with user-controlled posture per action	Selective disclosure exists; tier model and mixed-tier leak are not formalized	Theorem 12.2, Theorem 12.5
zk-KYC: ZK proof of credential-issuer-set membership without holder disclosure	Possible to construct on VC primitives; no normative profile	Section 12 § 12.8

Table 6: Treatise primitives positioned against W3C VC 2.0 and DIDs 1.1 as published through May 2026. The treatise is forward-compatible with the W3C stack at the wire-protocol layer (OID4VP/VCI, SD-JWT VC, ISO mdoc) and extends it at the model layer with post-quantum signatures, formal strata, hypergraph identity state, physical-good integration, and the seven principal theorems summarized in Section 14.

Layer	Holds	Speed	Mutability
QuantumHarmony chain	Identity records, hashes, CIDs, commitments, signatures	Slow	Append-only
IPFS	Encrypted payloads, KYC bundles, vault ciphertext	Medium	Immutable (b
Postgres + Redis	Denormalized indexes, query caches	Fast	Rebuildable n

Table 7: The three storage layers of the Paraxiom identity stack.